INNOPOLIS UNIVERSITY

SNE MASTER TRACK

COURSE: [F25] SECURITY OF SYSTEMS AND NETWORKS

SSN Lab 2 Assignment

On

OpenVPN Server

Student Name: HASIB AL TAHSIN
Application ID: 187623
Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 06.10.2025

Task 1 - Introduction

1a. What is PKI, and what is the purpose of PKI with OpenVPN?

b. Distinguish between a master certificate authority (CA) and a separate certificate authority (CA).

NB: Each client has a corresponding private key for the separate certificate, the public key. The primary Certificate Authority signs the server and client certificates.

Task 1

Solution:

1a. What is PKI, and what is the purpose of PKI with OpenVPN?

Solution:

PKI

A public key infrastructure (PKI) is a security system that manages public and private keys and verifies identities using digital certificates. It ensures that a public key actually belongs to the person to whom it is registered and maintains security, confidentiality, and trustworthiness during information exchange [1], [2].

The Purpose of PKI with OpenVPN

The main purpose of using Public Key Infrastructure (PKI) in OpenVPN is to ensure the authentication of both parties by establishing a secure connection. That is, both the client and the server verify each other's certificates to prevent fraud or fake connections, which is called Mutual TLS Authentication [3]. In this process, each user or server is issued a unique digital certificate, which proves their legitimacy. As a result, only authorized users can connect to the VPN server, which serves as the first step in security [4].

PKI ensures confidentiality, trust, and scalability for OpenVPN. The encryption process is completed with key pairs created through certificates, so that data is protected during communication [5]. A Certificate Authority (CA) acts as a central trusted source; certificates signed by the CA establish trust between both parties [6]. At the same time, the PKI framework is designed to easily create and manage multiple client certificates, and to quickly revoke a certificate if its security is compromised [7].

1b. Distinguish between a master certificate authority (CA) and a separate certificate authority (CA).
NB: Each client has a corresponding private key for the separate certificate, the public key. The primary Certificate Authority signs the server and client certificates

Solution:

The Master CA (or Root CA) is the certificate authority with the highest level of trust. It usually holds a self-signed certificate and is able to directly authorize any CA. Because it is the foundation of the system, its private key is kept very secret and protected. If this root CA is hacked or compromised, the entire PKI structure can be compromised [8], [9]. In many cases, the root CA is kept online and activated only when needed (offline root), only used when signing the certificates of subordinate CAs or creating CRLs (Certificate Revocation Lists) [9], [10].

On the other hand, Separate CAs (effectively multiple Subordinate or Intermediate CAs) are those CAs that work under the control of the main CA. Separate CAs usually issue client/server certificates, without the main CA acting directly on users. The separate CA has its own private key and certificate and receives a CA certificate signed by the main CA. This separation eliminates the need to keep the root CA's key online at all times and reduces the risk. If a separate CA's key or certificate becomes compromised, it can be revoked and the certificates issued by that CA can be disabled, but the root CA and other CAs will not be significantly affected [9], [10].

Differences

Master CA / Root CA	**Separate CA / Subordinate CA**
At the highest level, the main source of trust	Located in a chain trust, under the root CA
Self-signed or direct	CA certificate signed by the root CA
Maximum security, usually offline	Slightly lower risk, may be online during activities
If the root is damaged, the entire system is at risk	Only certificates under that CA will be affected

Task 2

Set-Up

OpenVPN enables mutual authentication using certificates, requiring both the client and server to verify each other's certificates before establishing trust. The client verifies the server's certificate, and the server does the same for the client. This is done by first confirming that the presented certificates were signed by the primary Certificate Authority (CA), followed by checking details in the authenticated certificate header, such as the common name or certificate type (client or server).

2a. Install, build, and initialize a Public Key Infrastructure for an installed OpenVPN.

b. What did you notice about the prompt when creating a CA? What is the reason behind creating and using a passphrase?

c. Generate Diffie Hellman parameters and key pair for the server. Show the location(path) of the key pair generated

d. What is the size of the Diffie Hellman parameters and their locations?

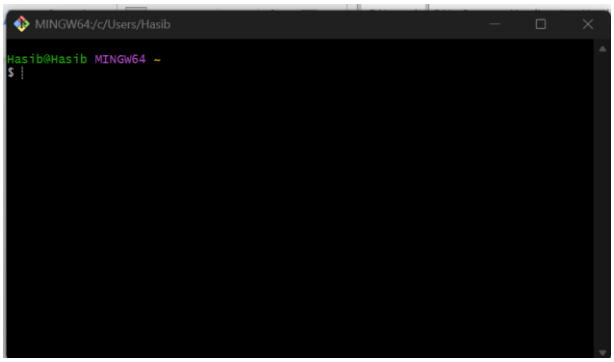
e. Create a certificate for the server. What is the commonName value, and how many days are the certificates signed on the server?

f. Create a key for the client and also a client certificate. What is the expiration date of the certificate?

2a: Install, build, and initialize a Public Key Infrastructure for an installed OpenVPN.

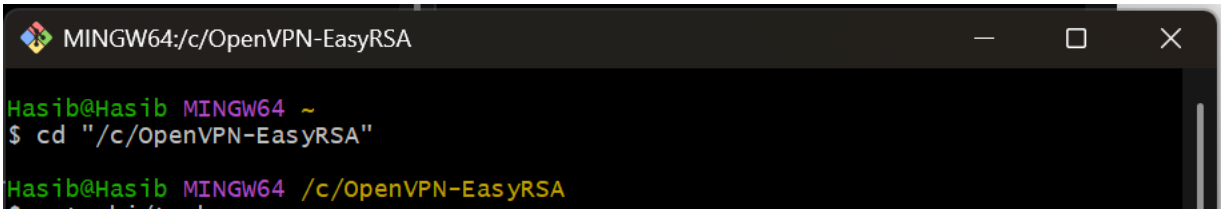
Solution:

OpenVPN was already installed on the computer, and then the EasyRSA ZIP file was downloaded and saved in the C:\EasyRSA-Start path. There, a folder called OpenVPN-EasyRSA existed, so it was attached [11]. Next, the Windows key was pressed to start the necessary configuration and Git Bash was typed and launched as Run as Administrator [12].

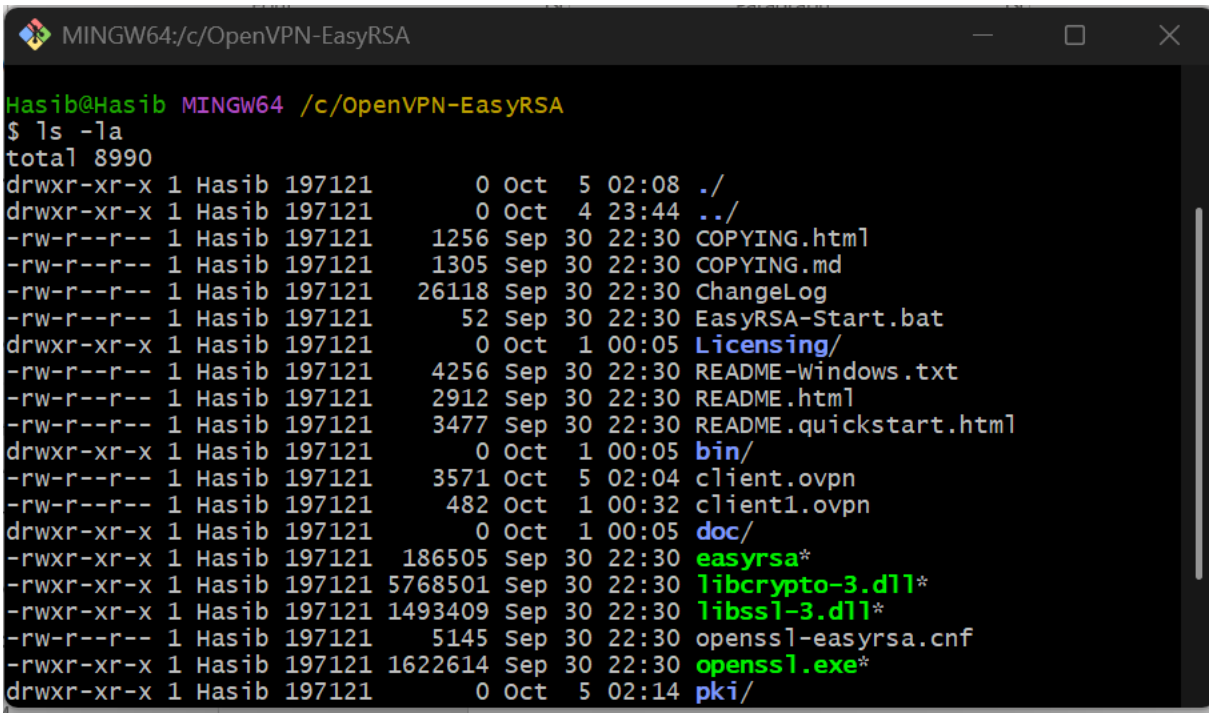


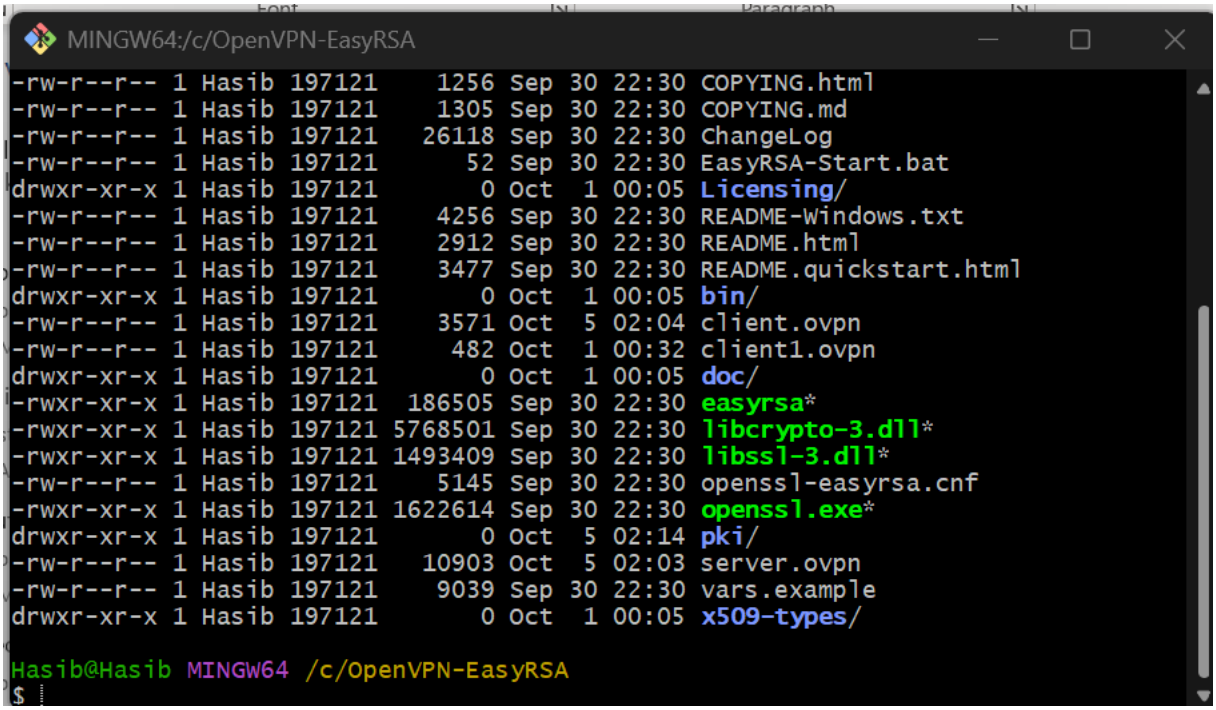
Getting to the right directory

In the initial step of using EasyRSA, after unzipping the ZIP file, all its contents were saved directly in the C:\EasyRSA-Start folder. Then, the following command was used to enter that directory from the command line (Git Bash):



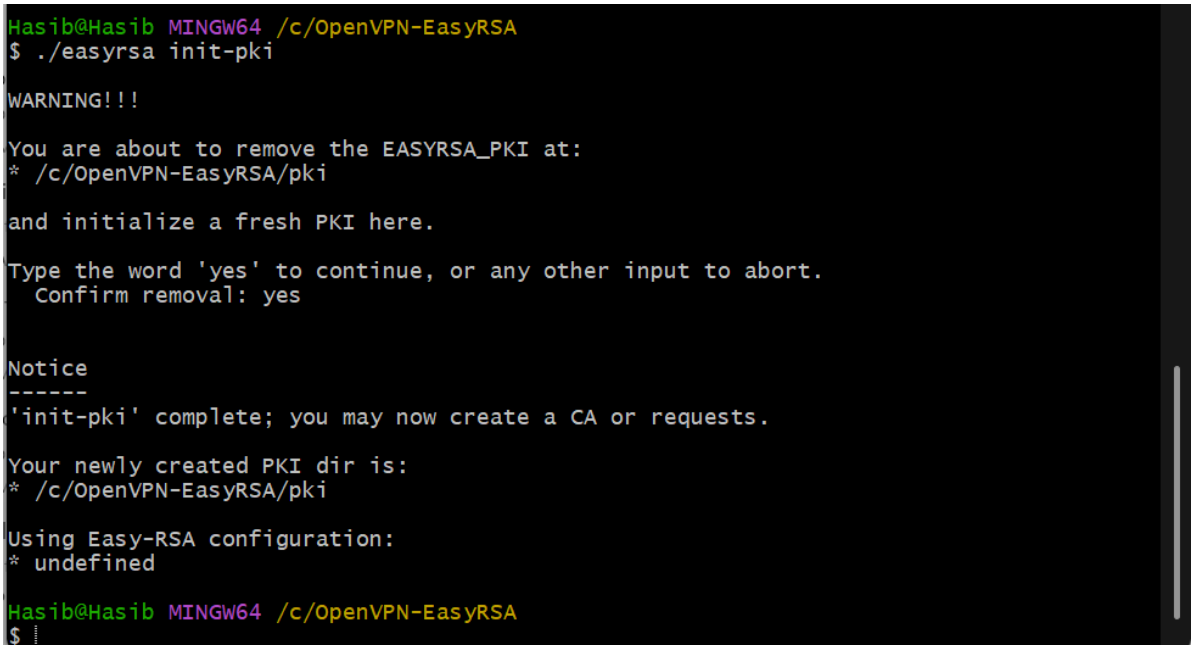
This command changed the working directory to C:\EasyRSA-Start. Then, to verify the files and folders inside the directory, typed the following command and pressed Enter, which displayed all the files in the corresponding folder, including hidden files, in detail [11], [13].





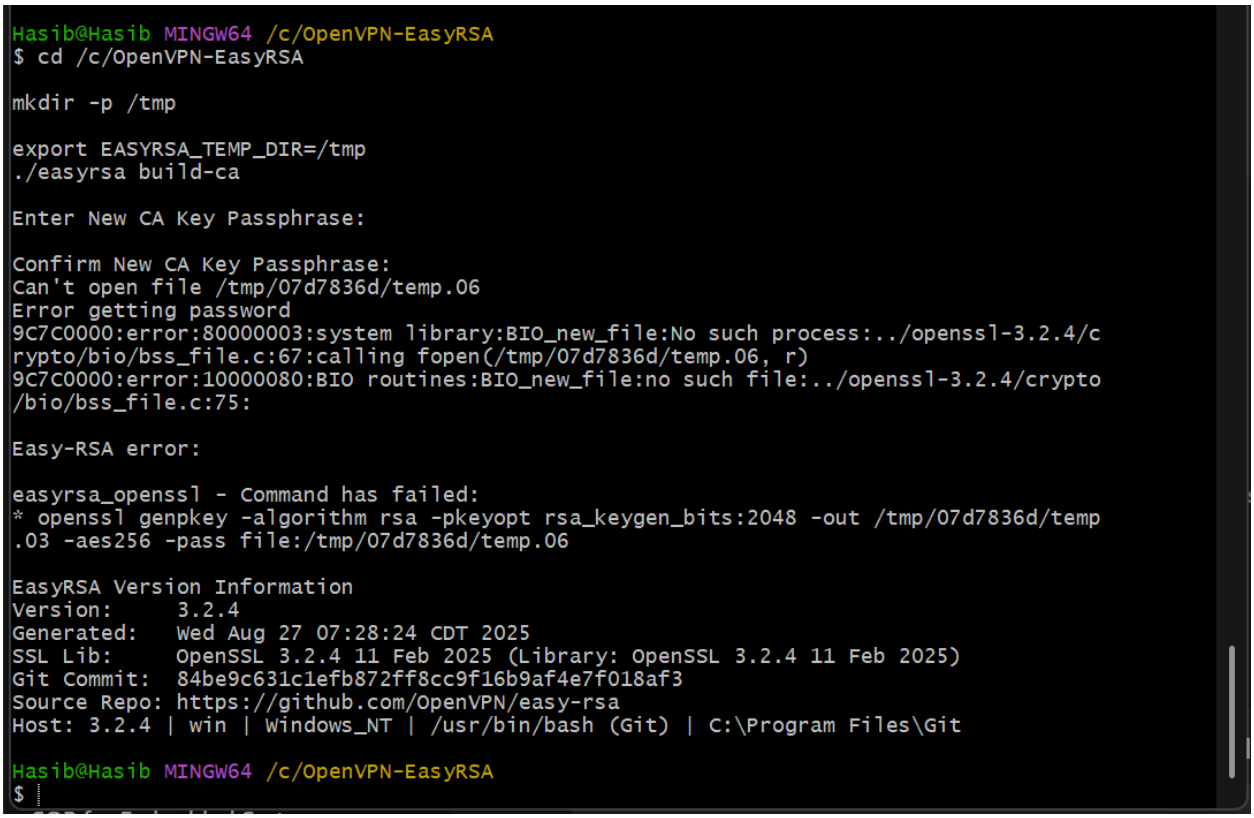
The zip contained some executables in OpenVPN-EasyRSA/bin/ and the output showed easyrsa or easyrsa script/bin/pki etc.

PKI init

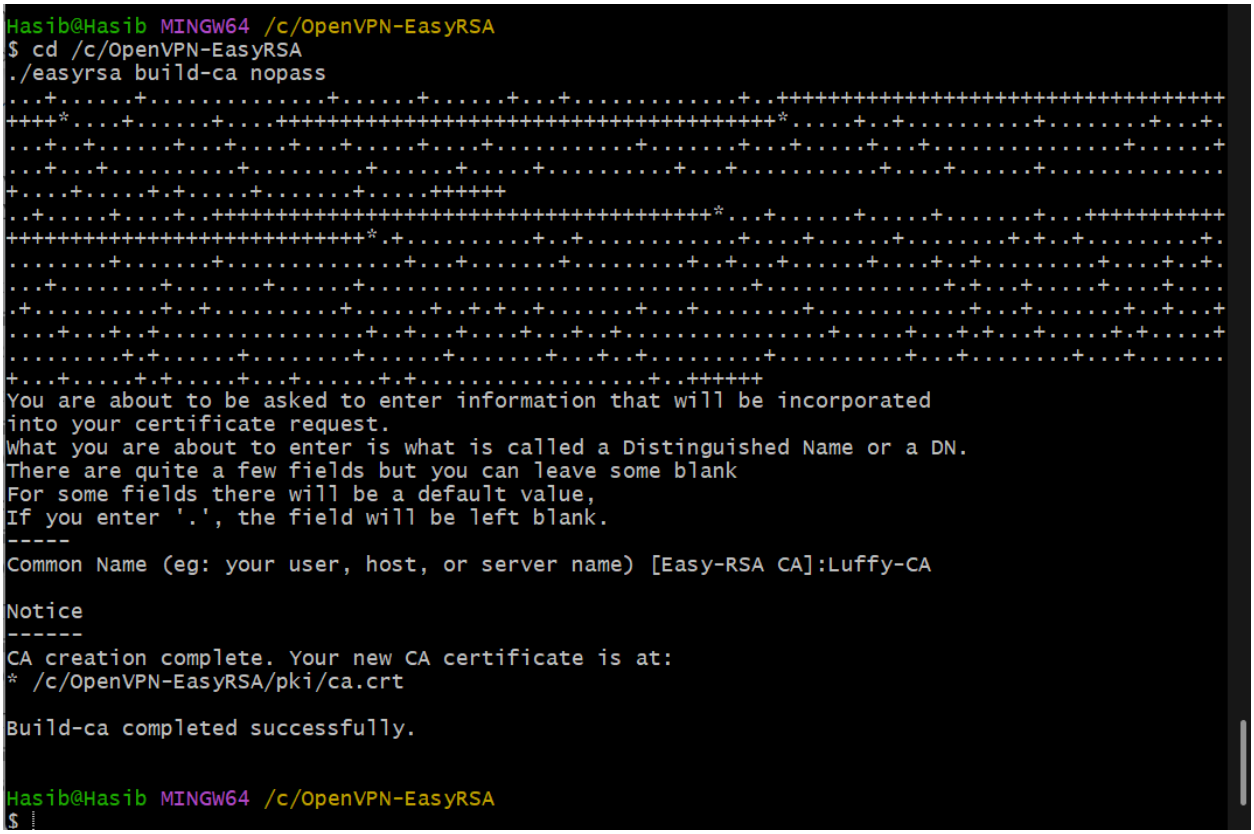


When running the command ./easyrsa init-pki using EasyRSA in Git Bash, it warned me to delete the previous PKI (Public Key Infrastructure) folder and create a new PKI. A WARNING appears on the screen, stating that the /c/OpenVPN-EasyRSA/pki folder will be removed and a new PKI will be created. **yes** was typed to confirm. After confirming, EasyRSA createed a new PKI directory /c/OpenVPN-EasyRSA/pki and stated that it was now possible to create a Certificate Authority (CA) or generate a certificate request. The EasyRSA configuration file currently appeared as undefined, which would be specified in the next setup [11], [13].

CA creation (interactive)



When trying to create a CA (Certificate Authority) using EasyRSA in Git Bash (./easyrsa build-ca), an error occured when creating a file in the /tmp directory. The output showed that OpenSSL could not find a temporary file and Error getting password indicated that the passphrase file could not be opened. Basically, EasyRSA was showing this problem because the /tmp directory did not exist on Windows. No passphrase was required when creating the CA and the /tmp error was avoided. This is a common solution for using EasyRSA in a Windows environment [13], [14].



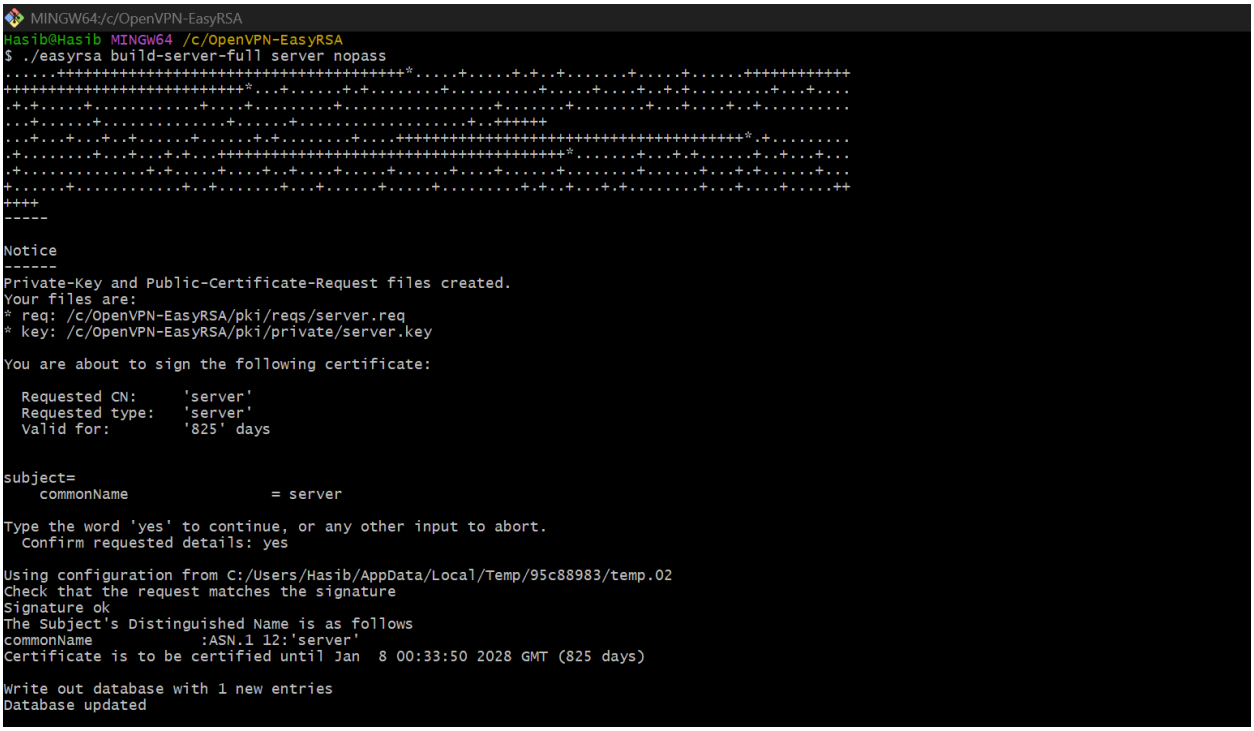
Using EasyRSA in Git Bash, a new Certificate Authority (CA) was created when the ./easyrsa build-ca nopass command was run. At the beginning of the process, the user was asked to provide some information, which was included in the certificate and was called the Distinguished Name (DN). Here, the Country, State, City, Organization, OU, and Common Name (CN) were required. The Common Name is typed as Luffy-CA. When the process was complete, the new CA certificate was saved in the file /c/OpenVPN-EasyRSA/pki/ca.crt and the Build-ca completed successfully message was displayed. As a general rule, it was safe to keep the CA's private key offline because it was highly sensitive [11], [14].

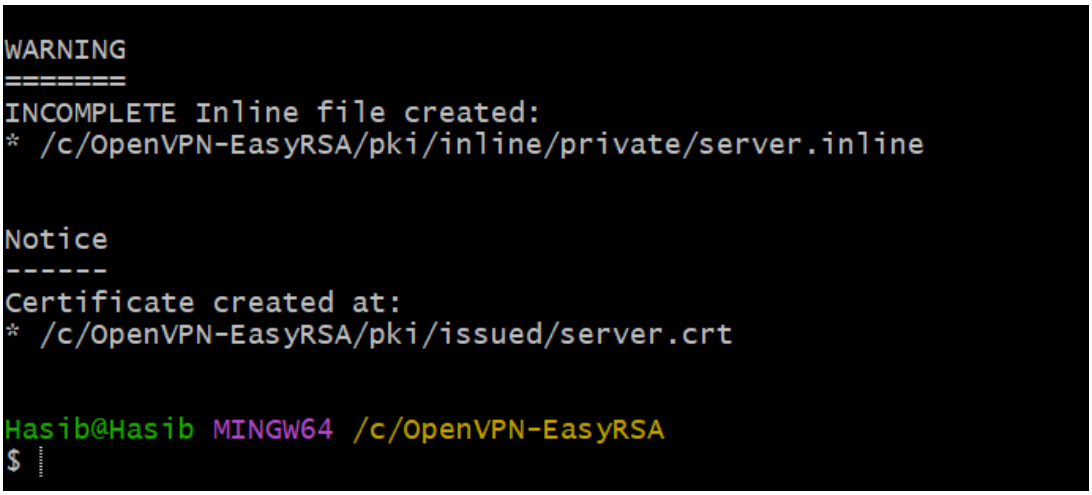
Server certificate & key

When generating a server certificate with EasyRSA, the server's name is usually given as the server's Common Name (CN). If the nopass option is used, the server's private key will not contain a passphrase. This is particularly convenient, as no manual password entry is required when the server boots automatically. After generating the certificate, two main output files were created: the server's public certificate pki/issued/server.crt and the private key pki/private/server.key. These

files are used in the OpenVPN server configuration and ensure the server's identity and security [14].

Client certificate





The full server certificate was generated by running the command ./easyrsa build-server-full server nopass in Git Bash. First, a private key and certificate request file were created,

pki/private/server.key and pki/reqs/server.req respectively. The user then had to type yes to verify the certificate details. The certificate was valid for 825 days after signing. At the end of the process, the new certificate was saved in the pki/issued/server.crt file. A warning was also displayed that some inline files are incomplete, which were created in the pki/inline/private/server.inline file. The server's private key did not have a passphrase. This was convenient for automatic booting [15], [16].

b. What did you notice about the prompt when creating a CA? What is the reason behind creating and using a passphrase?

Solution:

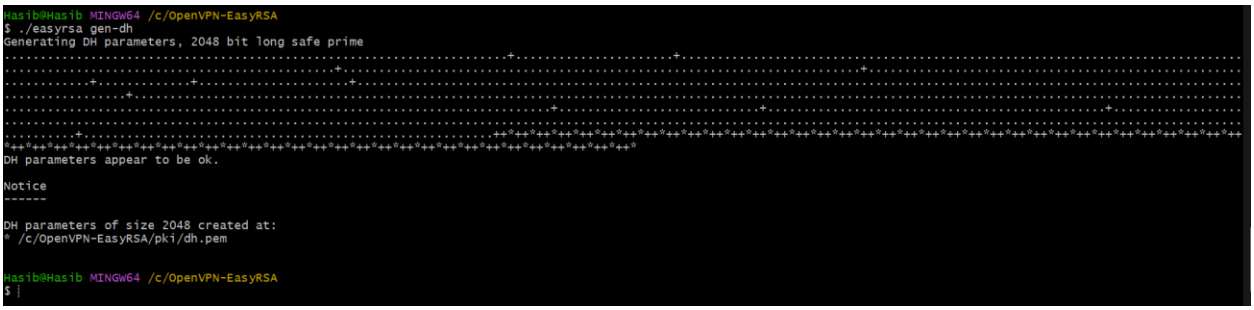
When running the ./easyrsa build-ca command, Git Bash asked me to provide some information step by step. First, I had to type the password of the CA private key at the Enter PEM pass phrase prompt; no characters appeared on the screen while typing. Then I had to type the same password again and pressed Enter. Then, I had to type the name of the CA at the Common Name (CN) prompt, for example, Luffy-CA. After this, I was asked for Country (C), State (ST), Locality (L), Organization (O), OU, Email, etc. step by step and I had to enter information in each field and pressed Enter [14].

The main purpose of using a passphrase is to ensure security. If the CA's private key (pki/private/ca.key) is encrypted with a passphrase, then even if someone steals the file, we will not be able to use it unless we know the passphrase. However, if the server's private key contains a passphrase, the server will not be able to start automatically at boot time; therefore, the server key is usually kept with nopass. However, the CA-key should be kept as secure as possible and stored offline [11].

2c: Generate Diffie Hellman parameters and key pair for the server. Show the location(path) of the key pair generated

Solution:

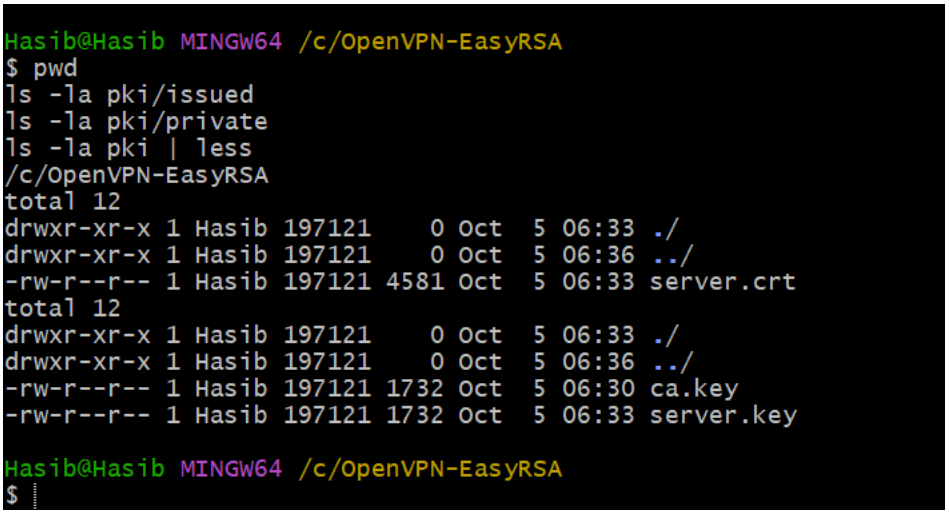
Creation of DH

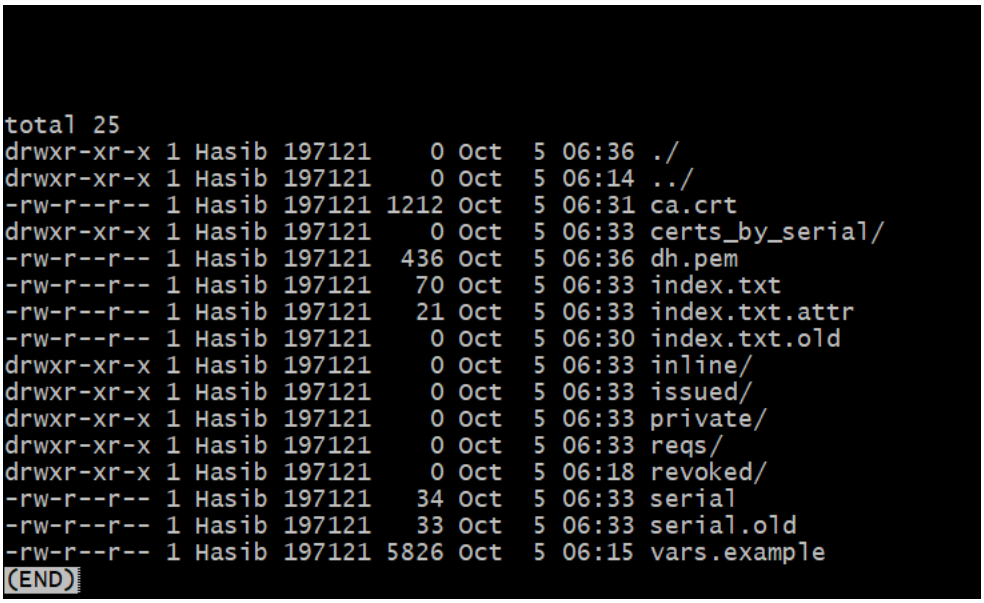


Running the ./easyrsa gen-dh command generated Diffie-Hellman (DH) parameters, which were used for secure confidentiality and key exchange in OpenVPN. This process taked some time to complete, especially when generating DHs of 2048-bit or larger sizes. When DH was successfully generated, it was saved in the pki/dh.pem file [16].

The server certificate and key had already been generated with ./easyrsa build-server-full server nopass, the main output files were: private key pki/private/server.key, public certificate pki/issued/server.crt, and DH parameters pki/dh.pem. In a Windows environment, the paths of these files were C:\EasyRSA-Start\OpenVPN-EasyRSA\pki\private\server.key, C:\EasyRSA-Start\OpenVPN-EasyRSA\pki\issued\server.crt, and C:\EasyRSA-Start\OpenVPN-EasyRSA\pki\dh.pem, respectively [17].

To verify my path in Git Bash





Using the pwd command in Git Bash displays the current directory path, such as /c/OpenVPN-EasyRSA. Then, using the ls -la pki/issued, ls -la pki/private, and ls -la pki | less commands, I saw the details of the files created by EasyRSA. The output showed that the server certificate server.crt was stored in the pki/issued folder, and the CA and server private keys (ca.key, server.key) were stored in the pki/private folder. In a Windows environment, replacing the /c/ path in Git Bash with just "C:" gives the same files, such as C:\EasyRSA-Start\OpenVPN-EasyRSA\pki\issued\server.crt and C:\EasyRSA-Start\OpenVPN-EasyRSA\pki\private\server.key [17].

2d: What is the size of the Diffie Hellman parameters and their locations?

Solution:

Using the pwd command in Git Bash displayed the current directory path, such as /c/OpenVPN-EasyRSA. Then, using the ls -la pki/issued, ls -la pki/private, and ls -la pki | less commands, I saw the details of the files created by EasyRSA. The output showed that the server certificate server.crt was stored in the pki/issued folder, and the CA and server private keys (ca.key, server.key) were stored in the pki/private folder. In a Windows environment, replacing the /c/ path in Git Bash with just "C:" gives the same files, such as C:\EasyRSA-Start\OpenVPN-

EasyRSA\pki\issued\server.crt and C:\EasyRSA-Start\OpenVPN-EasyRSA\pki\private\server.key [17].

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ openssl dhparam -in pki/dh.pem -noout -text | head -n 1
    DH Parameters: (2048 bit)
```

Here 2048 bits was the size of the DH-parameter, which indicated how large a prime number was used for the Diffie-Hellman key exchange. This size is recommended for secure encryption in OpenVPN and is generated by EasyRSA by default [14].

2e: Create a certificate for the server. What is the commonName value, and how many days are the certificates signed on the server?

Solution:

Creating a server certificate

Step 1:

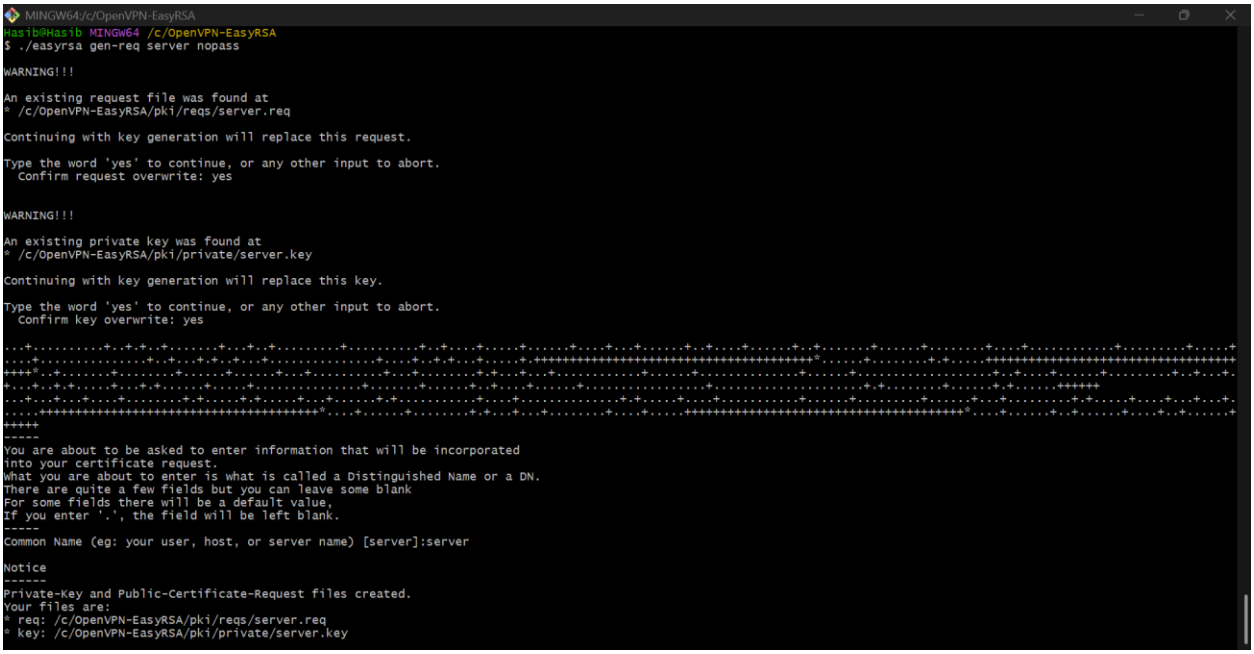
```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ ./easyrsa build-server-full server nopass

EasyRSA version 3.2.4

Error
-----
Request file already exists. Aborting build to avoid overwriting this file.
If you wish to continue, please use a different name.
Conflicting file found at:
* /c/OpenVPN-EasyRSA/pki/reqs/server.req
```

When running the ./easyrsa build-server-full server nopass command in Git Bash, EasyRSA will throw an error if a server request file (server.req) with the same name has already been created. This means that the previous request file already exists and EasyRSA will not overwrite it. To resolve the issue, I needed to use a new server name, or to delete the previous request/certificate and tried again. This is a security measure in EasyRSA to prevent the loss of old certificates and keys [18], [19].

Step 2: gen-req and sign-req



When running the ./easyrsa gen-req server nopass command in Git Bash, EasyRSA first checked for the presence of the previous request file (server.req) and private key (server.key). The files were already existed. It displayed two WARNINGs and asked for permission to overwrite. Typing yes overwrites the previous files and created a new private key and certificate request (CSR). Then, it asked for the Distinguished Name (DN), such as the Common Name (CN) of the server. Finally, a new request and key were created, stored in the files pki/reqs/server.req and pki/private/server.key, respectively. To validate the new server certificate, this CSR file needed to be signed by a CA. In this way, the server certificate (server.crt) was generated and could be used on the OpenVPN server [18], [19].

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ ./easyrsa sign-req server server
Please check over the details shown below for accuracy. Note that this request
has not been cryptographically verified. Please be sure it came from a trusted
source or that you have verified the request checksum with the sender.
You are about to sign the following certificate:

  Requested CN:     'server'
  Requested type:   'server'
  Valid for:        '825' days


subject=
    commonName                = server

Type the word 'yes' to continue, or any other input to abort.
  Confirm requested details: yes

Using configuration from C:/Users/Hasib/AppData/Local/Temp/cb5ac463/temp.02
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'server'
Certificate is to be certified until Jan  8 01:02:51 2028 GMT (825 days)

Write out database with 1 new entries
Database updated

WARNING
=======
INCOMPLETE Inline file created:
* /c/OpenVPN-EasyRSA/pki/inline/private/server.inline


Notice
------
Certificate created at:
* /c/OpenVPN-EasyRSA/pki/issued/server.crt
```

The server's certificate request (CSR) was signed by the CA by running the command ./easyrsa sign-req server server . It was first asked to verify the details of the CSR, such as the Requested CN (server) and the validity period (i.e. 825 days). Typing yes completed the EasyRSA certificate signing process. The certificate's signature was verified and the database was updated.

After successful signing, the certificate was created, along with a warning that some inline files were incomplete (pki/inline/private/server.inline). This certificate was usable on the OpenVPN server and ensured the server's identity and security [20], [21].

Verification

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ ls pki/issued/
ls pki/private/
server.crt
ca.key  server.key

Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

After generating a server certificate with EasyRSA, the output will produce several key files: server.crt, ca.key, and server.key. Here, the server name is used as the Common Name (CN), which is included in the certificate as the server's identity. The certificate (server.crt) provides the server's public identity, the private key (server.key) is stored securely, and the CA private key (ca.key) is used for CA control and signing [18].

How to view CN

The output after the server certificate was generated in EasyRSA was as follows:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ openssl x509 -in pki/issued/server.crt -noout -subject
subject=CN=server

Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

CN or Common Name was the name of the server which was included in the certificate. It determines the identity of the server and is used to establish a trusted connection between the client and the server on the OpenVPN server. CN was the name by which the server was identified [20].

How long was **the certificate valid (signed)**:

Step A: Check the validity period in the certificate file:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ openssl x509 -in pki/issued/server.crt -noout -dates
notBefore=Oct  5 01:02:51 2025 GMT
notAfter=Jan  8 01:02:51 2028 GMT

Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

Running this displayed the validity period of the server's certificate. Here notBefore was the date and time the certificate was activated and notAfter was the date and time the certificate expires. This means that the certificate will be valid from October 5, 2025, to January 8, 2028 [21].

Step B: Calculate the number of days:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ nb=$(openssl x509 -in pki/issued/server.crt -noout -dates | sed -n 's/notBefore=//p')
na=$(openssl x509 -in pki/issued/server.crt -noout -dates | sed -n 's/notAfter=//p')
nb_s=$(date -d "$nb" +%s)
na_s=$(date -d "$na" +%s)
echo $(( (na_s - nb_s) / 86400 )) days
825 days

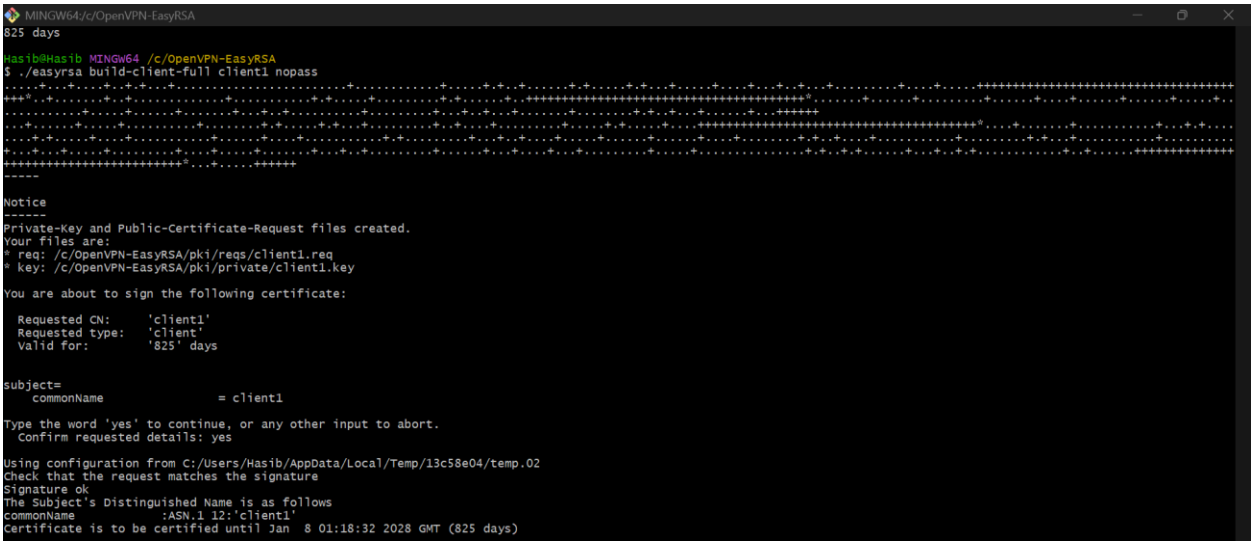
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

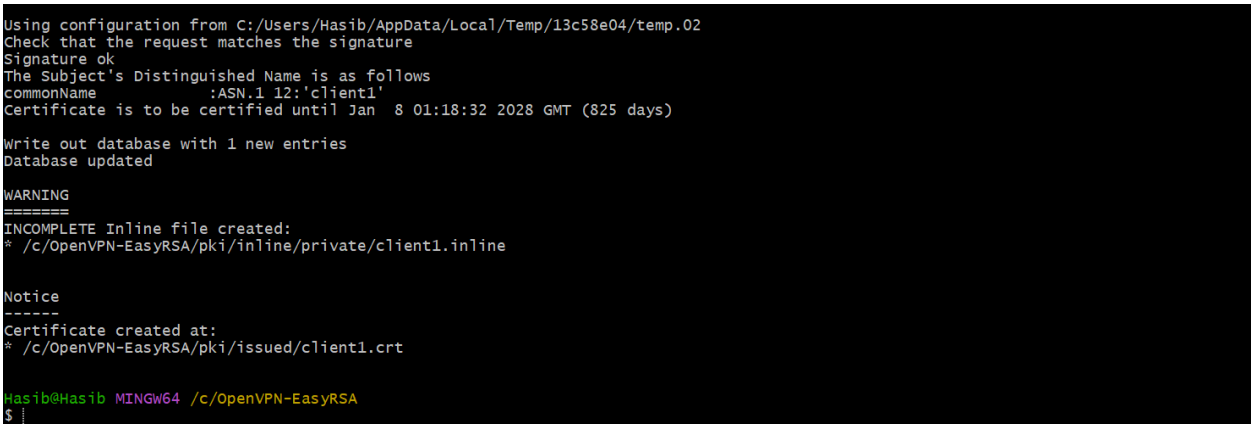
This indicates that the certificate will be valid for 825 days. The certificate expiration in EasyRSA was usually specified by the EASYRSA_CERT_EXPIRE variable, which was set to 825 days by default [22].

2f: Create a key for the client and also a client certificate. What is the expiration date of the certificate?

Solution:

Client and certificate creation





A new private key and certificate were generated for client1 by running the client1 nopass command. First, a CSR (certificate request) and private key were generated in Git Bash, in the files pki/reqs/client1.req and pki/private/client1.key, respectively. The user was then asked to verify the certificate details, such as the Common Name (CN) of client1 and the validity period (825 days). The user as myself typed yes, the CSR is signed by the CA. After successful signing, the certificate was generated and saved in the file pki/issued/client1.crt. A warning was also displayed that some inline files were incomplete (pki/inline/private/client1.inline) [23].

How to check the expiration date of a client certificate

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ openssl x509 -in pki/issued/client1.crt -noout -dates -subject
notBefore=Oct  5 01:18:32 2025 GMT
notAfter=Jan  8 01:18:32 2028 GMT
subject=CN=client1
```

Running the above command displayed the validity period and subject of the client1 certificate. Here notBefore was the time the certificate was activated and notAfter was the time the certificate expired. It indicates the certificate expiration date. In the subject, I saw that client1 was used as the CN (Common Name), which identified the client.

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ exp=$(openssl x509 -in pki/issued/client1.crt -noout -dates | sed -n 's/notAfter=//p')
exp_s=$(date -d "$exp" +%s)
now_s=$(date +%s)
echo $(( (exp_s - now_s) / 86400 )) days remaining
824 days remaining

Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

The above commands in Git Bash can be used to determine the remaining days of a certificate's validity. The above output showed that the certificate would expire after 824 days from the current date. This means that this command is used to determine the remaining validity of the certificate [24].

The CA-key was kept safe. If the CA-key is leaked, the entire PKI will be broken. The CA private key should generally be kept offline. I copied the following files to the OpenVPN server config.

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ cp pki/ca.crt "/c/Program Files/OpenVPN/config/ca.crt"
cp pki/issued/server.crt "/c/Program Files/OpenVPN/config/server.crt"
cp pki/private/server.key "/c/Program Files/OpenVPN/config/server.key"
cp pki/dh.pem "/c/Program Files/OpenVPN/config/dh.pem"
```

Then ca ca.crt, cert server.crt, key server.key, dh dh.pem were specified in server.ovpn or server.conf.

Task 2: Part 2

3a: What is the TLS Authentication (TA) Key, and why is it essential in OpenVPN?

Solution:

TLS Authentication (TA) Key

The ta.key or TLS Authentication key is a fixed pre-shared key (HMAC secret) that attaches an HMAC signature to packets on the TLS control channel of OpenVPN. The server only accepts packets whose HMAC is valid, so that unauthorized or malicious traffic cannot reach the server. This increases the initial level of security on the VPN server, because only clients with the correct ta.key can complete the TLS handshake [1].

Why it is important:

- **DoS/Port-Scan Prevention:** If any unauthorized tool or scanner sends TLS handshake packets, the server discards them if the ta.key is not verified. This reduces server resource consumption and prevents DoS attacks [25].
- **Initial Authentication:** The server verifies the HMAC before the TLS handshake begins. Only clients with the correct ta.key can complete the TLS handshake, which provides additional initial security [25].
- **Different from others:** OpenVPN's tls-crypt (version 2.4+) is more robust than tls-auth, as it also encrypts the control channel. However, ta.key (tls-auth) is still simpler and more widely used, especially in simple configurations [25].

3b: Generate TLS Authentication (server) and show the server key information

Solution:

Preparation

I typed to Git Bash and clicked Run as administrator to enter Git Bash**.**

Step 1: Go to EasyRSA folder

The folder is C:\OpenVPN-EasyRSA. I typed in Git Bash:

cd "/c/OpenVPN-EasyRSA"

Step 2: Check if pki/ta.key exists or not

Add OpenVPN path to Git Bash

I opened Git Bash and typed:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ export PATH=$PATH:"/c/Program Files/OpenVPN/bin"
```

Now checked:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ openvpn --version
OpenVPN 2.6.15 [git:v2.6.15/90bdd59a95170169] Windows [SSL (OpenSSL)] [LZO] [LZ4] [PKCS11] [AEAD] [DCO] built on Sep 22 2025
library versions: OpenSSL 3.5.3 16 Sep 2025, LZO 2.10
Windows version 10.0 (Windows 10 or greater), amd64 executable
DCO version: 1.3.3
Originally developed by James Yonan
Copyright (C) 2002-2024 OpenVPN Inc <sales@openvpn.net>
Compile time defines: N/A
```

Running the openvpn --version command in Git Bash displays the OpenVPN version and associated library information. The output showed that OpenVPN 2.6.15 was being used, compiled with OpenSSL 3.5.3, LZO 2.10 on Windows. It also displayed the device architecture (amd64), DCO version (1.3.3), and other compile-time properties [25].

Step 3: Create TLS auth key

Once the path was correct, I typed in Git Bash:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ openvpn --genkey --secret pki/ta.key
2025-10-05 07:57:45 DEPRECATED OPTION: The option --secret is deprecated.
2025-10-05 07:57:45 WARNING: Using --genkey --secret filename is DEPRECATED.  Use --genkey secret filename instead.
```

Running this command displayed detailed information about the OpenVPN installation and version. The output showed that OpenVPN 2.6.15 was currently being used, which runed on Windows and supports OpenSSL 3.5.3 libraries, LZO, LZ4 compression, PKCS11, and other

features. It also displayd DCO (Data Channel Offload) version 1.3.3 and other compile-time information. This command is primarily used to verify the OpenVPN version and build environment [26].

Step 3: Checking the location of ta.key and checking the small part for the server

Verifying the location of ta.key by creating it:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ ls -la pki/ta.key
-rw-r--r-- 1 Hasib 197121 657 Oct  5 07:57 pki/ta.key

Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

Running the above command in Git Bash showed the details of the ta.key file. The output showed that the file is 657 bytes in size, and was stored in the pki directory. This is the TLS Authentication key for OpenVPN, which is used for HMAC signing. The file permissions -rw-r--r-- indicate that I can read and write to it, while others can only read it. Using this file, the server only accepts valid TLS control channel packets.\

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ cat pki/ta.key
#
# 2048 bit OpenVPN static key
#
-----BEGIN OpenVPN Static key V1-----
e5c72a90a7d4d5fd70d4824c562254fb
f95f43e7defa0998d414408dbf9e7454
9eb78963f10052e226a32b9078477133
93e0023ad2fd74d8105602be00838196
b381053d2c5a74ce837d6f6e640df48b
f96172a0556f419e1f79187faaa5f9e2
39696264473793f0c28afdcd7c7282e5
1e5584a3c9ad8dcba8b8a41f466096c8
4cc51f3ba0e6e86be4d770dea68377c7
b8401a0de496966b338c3a6a2ff0e266
b147878c86bb90d56dc6c0ce206ee8d7
8361a075f1677b8dc764ba281f820dfe
3be3212c765f1730a731bf798d6008bb
e1ef2f88ed3ee956d2e89973d30c7bf2
680226f0f44fc58c90dde33a22b450fb
2ae629eac1e9a521d588c1a8b58fa408
-----END OpenVPN Static key V1-----

Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$
```

The full contents of the ta.key file was displayed. The output showed a 2048-bit OpenVPN Static Key, which was used for HMAC signing. The file has -----BEGIN OpenVPN Static key V1-----

and -----END OpenVPN Static key V1----- markers at the beginning and end. Using this key, the server only accepts valid TLS control channel packets, discarding unauthorized packets.

Step 5: Copying to the server's OpenVPN config folder

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ cp pki/ta.key "/c/Program Files/OpenVPN/config/ta.key"
```

Step 6: Adding to Server config (server.ovpn / server.conf)

To open server.ovpn with Notepad on Windows: Opened Notepad → File → Opened → Browse and selected C:\Program Files\OpenVPN\config\server.ovpn.

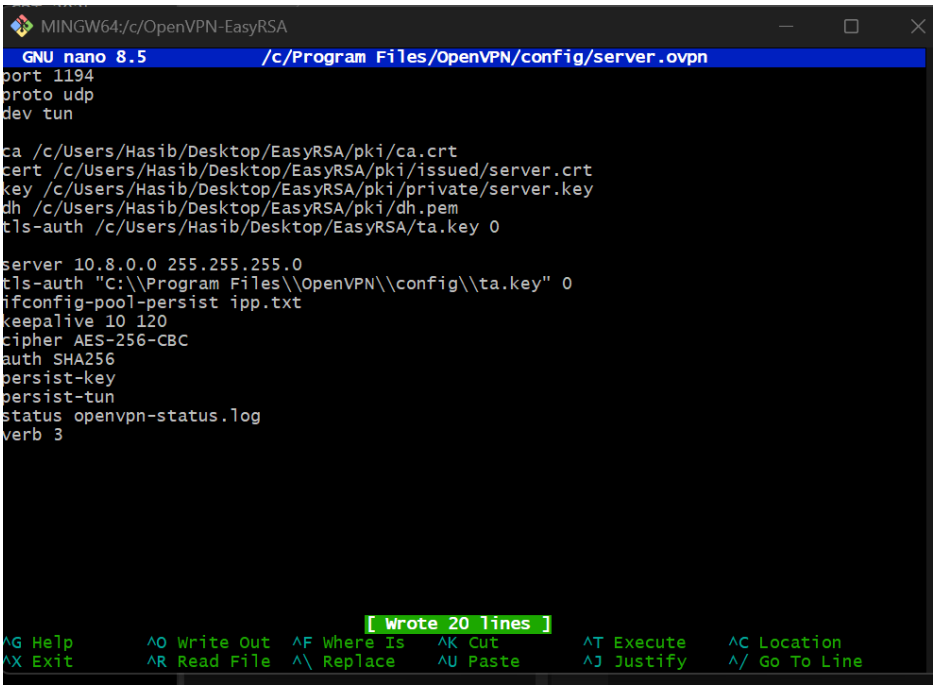
Opening and editing server.ovpn

To open and edit the server.ovpn file using an editor in Git Bash:

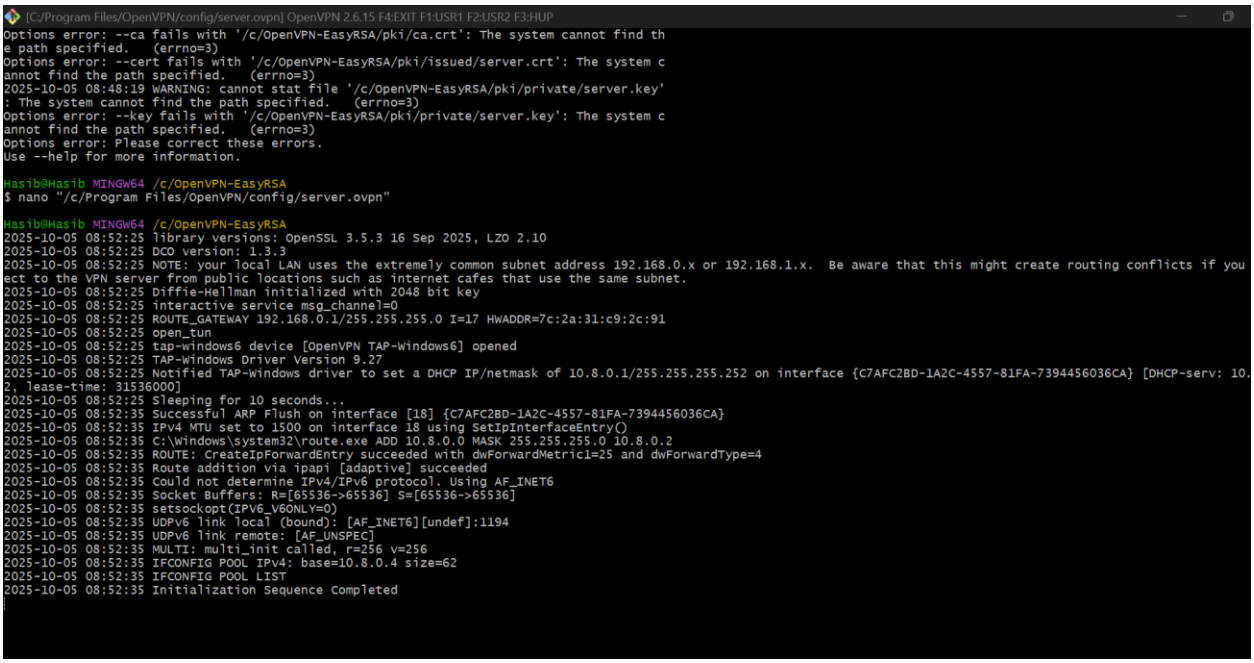
```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ nano "/c/Program Files/OpenVPN/config/server.ovpn"
```

Fixed the tls-auth line

I found it in server.ovpn and changed it to:



I **tried again in Git Bash:**

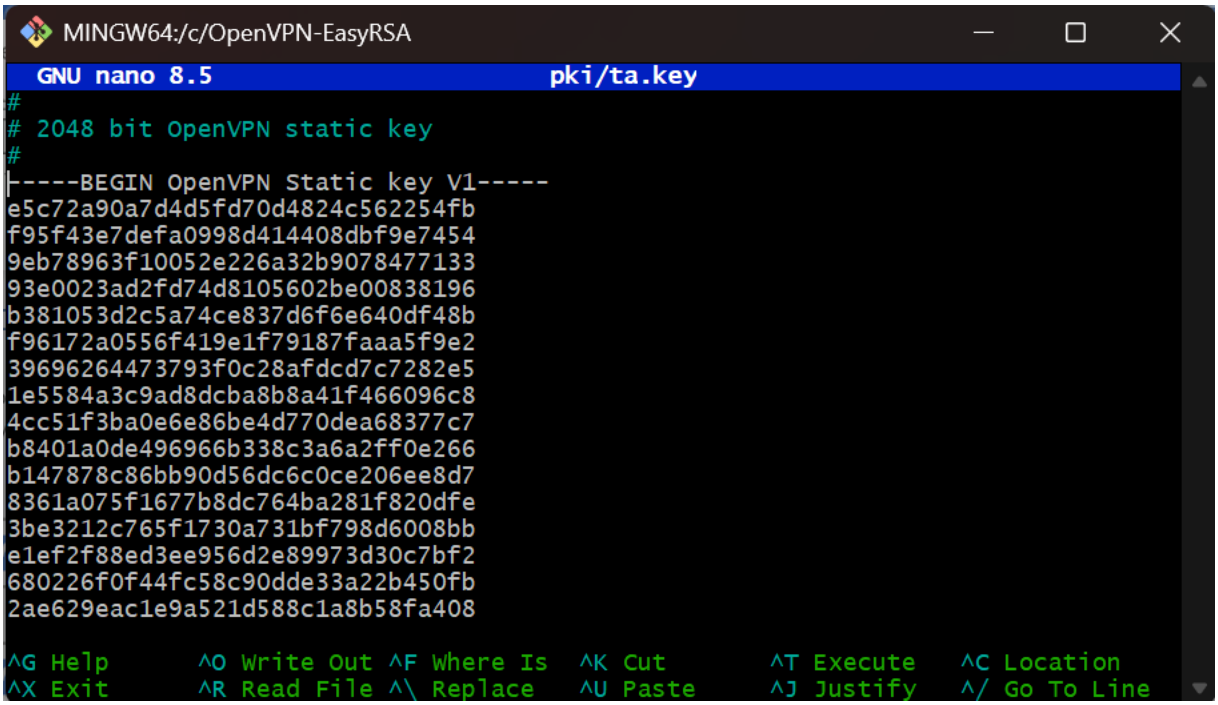


After running the command in Git Bash, the server.ovpn configuration file was opened in the nano editor. The output showed the log of the OpenVPN server startup. According to the log, OpenVPN had initialized the TLS session using a 2048-bit Diffie-Hellman key, the TAP-Windows6 device had been successfully opened, and the DHCP IP/netmask (10.8.0.1/255.255.255.252) had been set correctly. The server routing and IP forward configuration had been successfully applied, and the IPv4/IPv6 socket had been prepared. The IFCONFIG pool has been created and the Initialization Sequence has completed [27].

3c: Generate a TLS Authentication key (client) and show the client key information

Solution:

Usually the same ta.key is used - the same file has to be kept on the server and client (copied via secure channel). Since they are created separately, they will not work with each other. So a new ta.key is not created separately for the client, but the server's pki/ta.key is copied to the client.



The output showed a 2048-bit OpenVPN Static Key. It is part of the ta.key file and is used for the TLS Authentication (tls-auth) feature of OpenVPN. The file generates an HMAC signature, which ensures that the server only accepts valid TLS control channel packets.

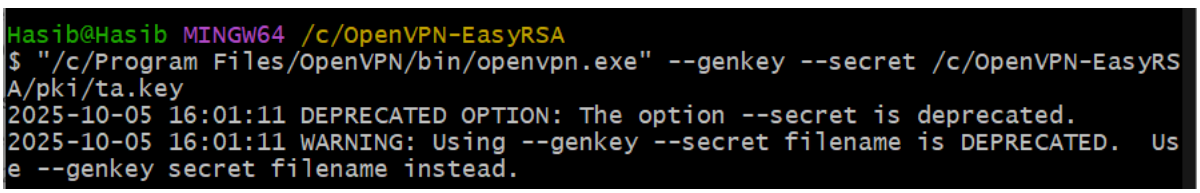
Copy the ta.key file to the client config folder

The command in Git Bash was:



The file /c/OpenVPN-EasyRSA/pki/ta.key did not exist, so I need to generate ta.key first.

To generate ta.key in OpenVPN EasyRSA:



It appeared that the --secret option was deprecated. The log showed a warning: Use --genkey secret filename instead. That is, in newer versions of OpenVPN, I should use --genkey secret filename instead of --genkey --secret filename [28].

Adding the line to client.ovpn

The command in Git Bash was:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ echo -e "\ntls-auth \"C:\\Program Files\\OpenVPN\\config\\ta.key\" 1\nkey-dire
ction 1" >> "/c/Program Files/OpenVPN/config/client.ovpn"
```

Checking the client.ovpn file in Git Bash:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ tail -n 5 "/c/Program Files/OpenVPN/config/client.ovpn"
key-direction 1

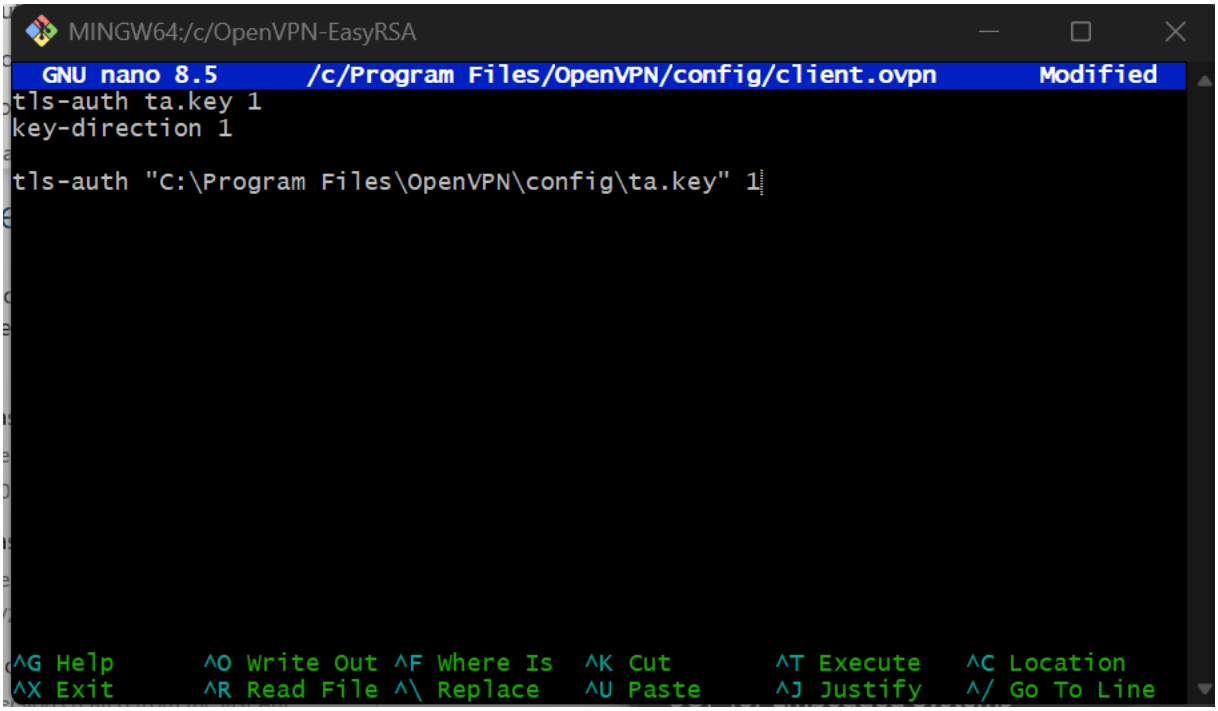
tls-auth "C:\Program Files\OpenVPN\config\ta.key" 1

tls-auth "C:\Program Files\OpenVPN
```

Running the command in Git Bash displayed the last 5 lines of the client.ovpn file. The output showed the lines related to tls-auth. This confirms that the TLS Authentication key (ta.key) has been correctly added to the client configuration. The key-direction 1 indicates that the direction of key usage between the server and the client is correctly configured. This will allow the OpenVPN client to verify the HMAC for the TLS control channel and block mismatched traffic [28].

Inline embed - so I can keep everything in a single .ovpn file:

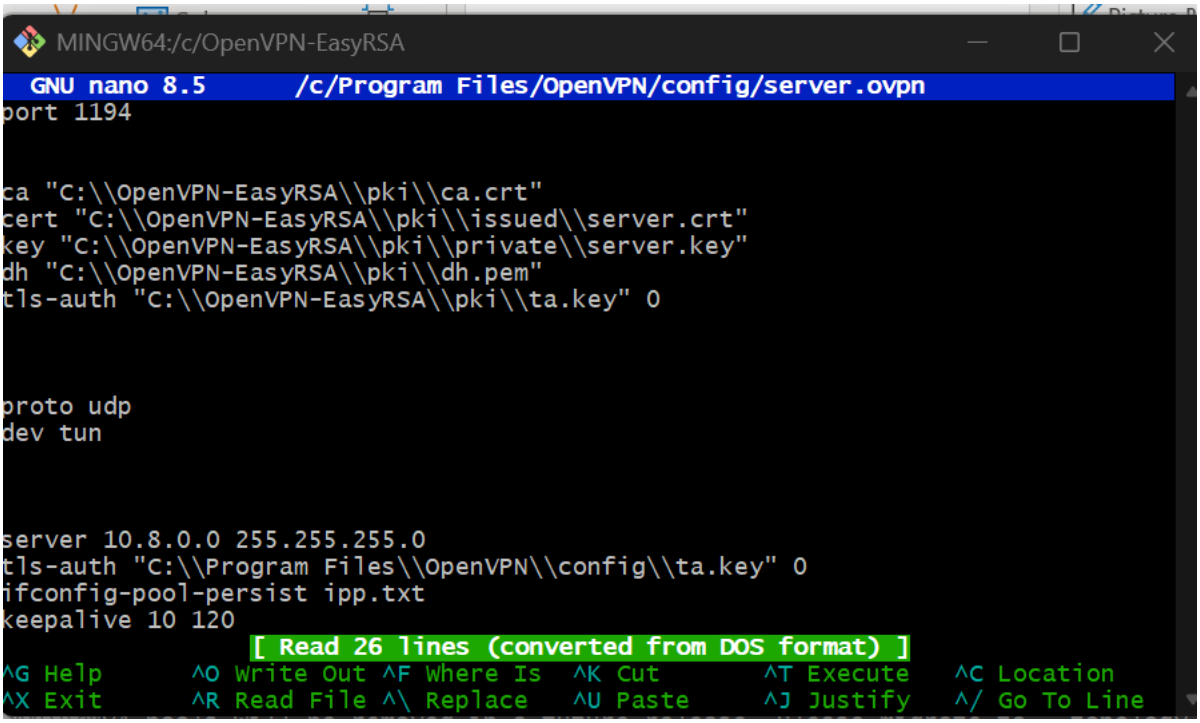
Many clients support inline <tls-auth>. If the client supports inline, I pasted it into client.ovpn like this:



In OpenVPN, tls-auth ta.key 1 and key-direction 1 ensure client/server TLS control channel HMAC verification. The key is loaded with the Windows path [29].

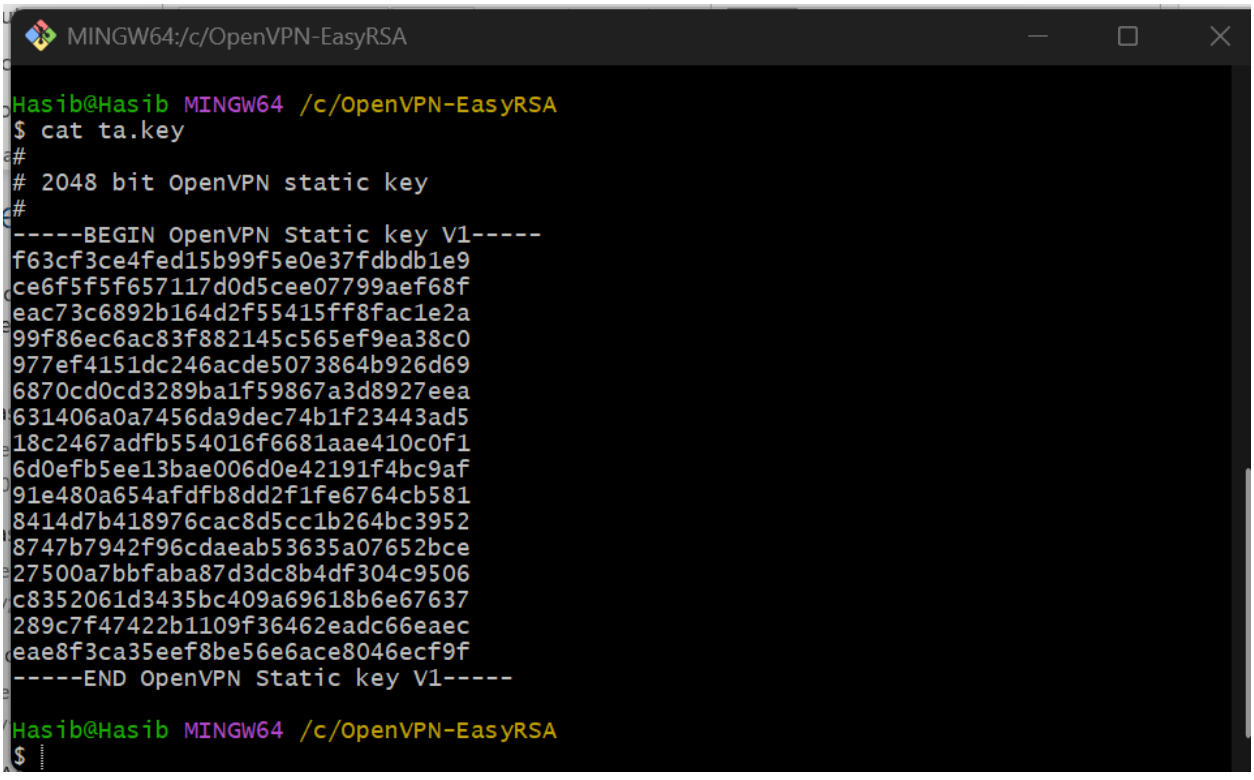
```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ nano "/c/Program Files/OpenVPN/config/server.ovpn"
```

This OpenVPN server configuration script runs the server on UDP port 1194 with port 1194 and proto udp. TLS authentication (tls-auth) and PKI files (ca.crt, server.crt, server.key, dh.pem) were included. TUN device was used with dev tun, IP range was set to 10.8.0.0/24, ifconfig-pool-persist ipp.txt client IP storage and keepalive 10 120 connections were monitored [29].



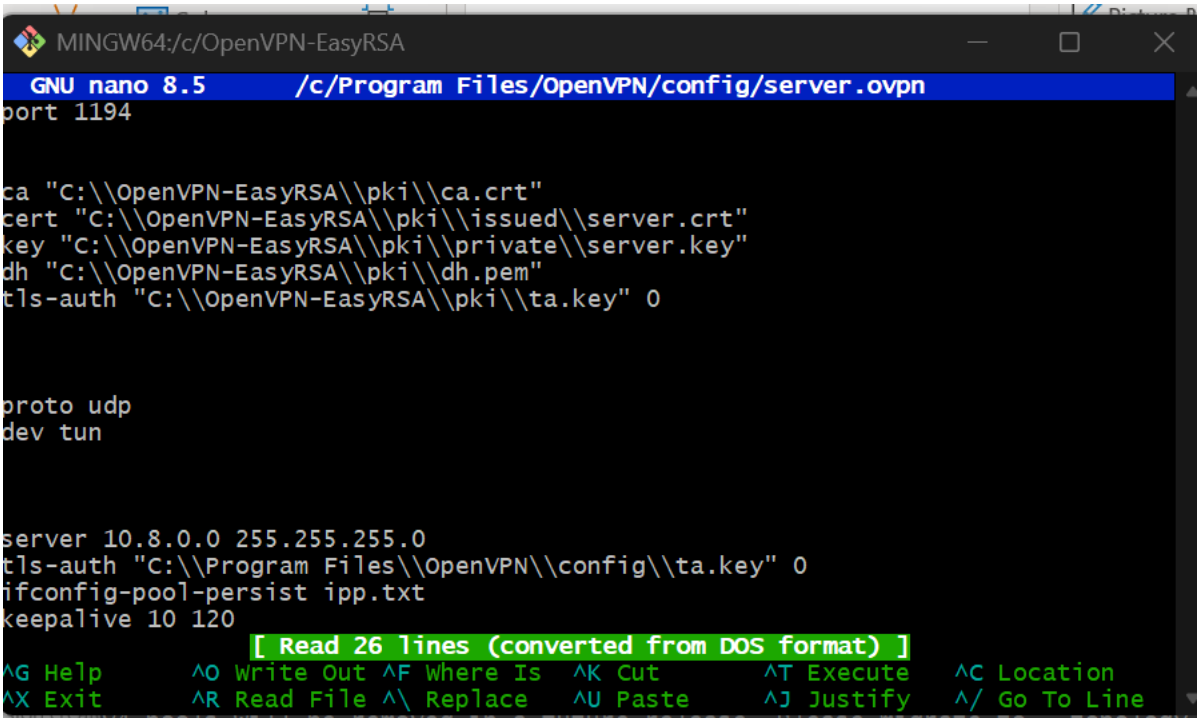
Step 3: View key info on the client (client side):

cat ta.key on the client

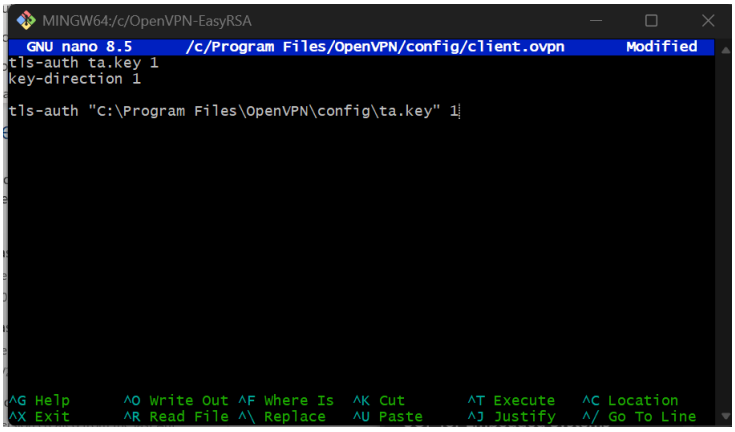


Running the above command in Git Bash will output the first few lines of the ta.key file. This shows the “BEGIN OpenVPN Static key V1” block and the initial part of the 2048-bit OpenVPN static key, which is used for TLS authentication [25].

I added to server.ovpn:

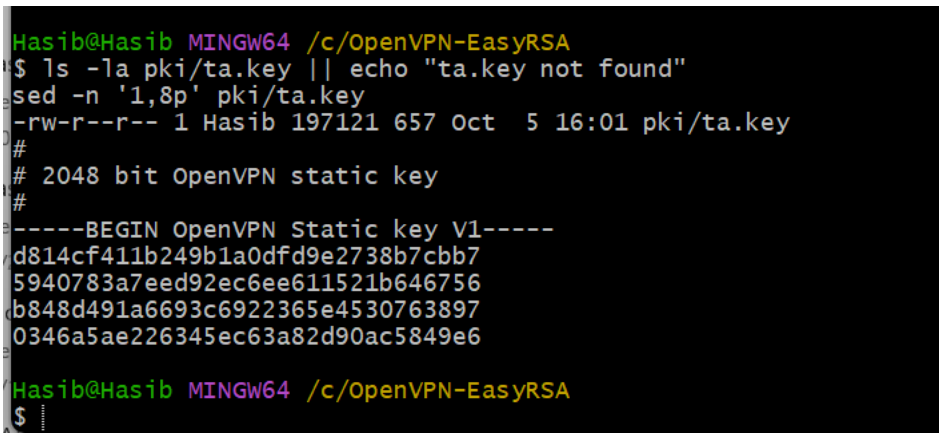


To set up TLS Authentication on the OpenVPN client, the same ta.key file was copied using a secure channel (such as SCP, USB, or encrypted email). Then, the <tls-auth> inline block was added to the client.ovpn config file, so that the TLS control channel packets could be HMAC signed. The key direction is set using the tls-auth ta.key 1 and key-direction 1 directives in the client config. This ensures that only clients with the correct ta.key can complete the TLS handshake with the server, thus blocking unauthorized traffic.



Then I pressed Ctrl + o, pressed Enter, saved, and pressed Ctrl + x to exit the editor.

To test whether ta.key was present on the machine, ran (in Git Bash):



The command in Git Bash showed that the pki/ta.key file existed and was 657 bytes in size. Also, the first 8 lines showed the “2048 bit OpenVPN static key” block and BEGIN OpenVPN Static key V1.

Task 3 - Verification

3a. Simulate a communication between the OpenVPN server and the and client.

Solution:

Changing to the EasyRSA folder using the command:

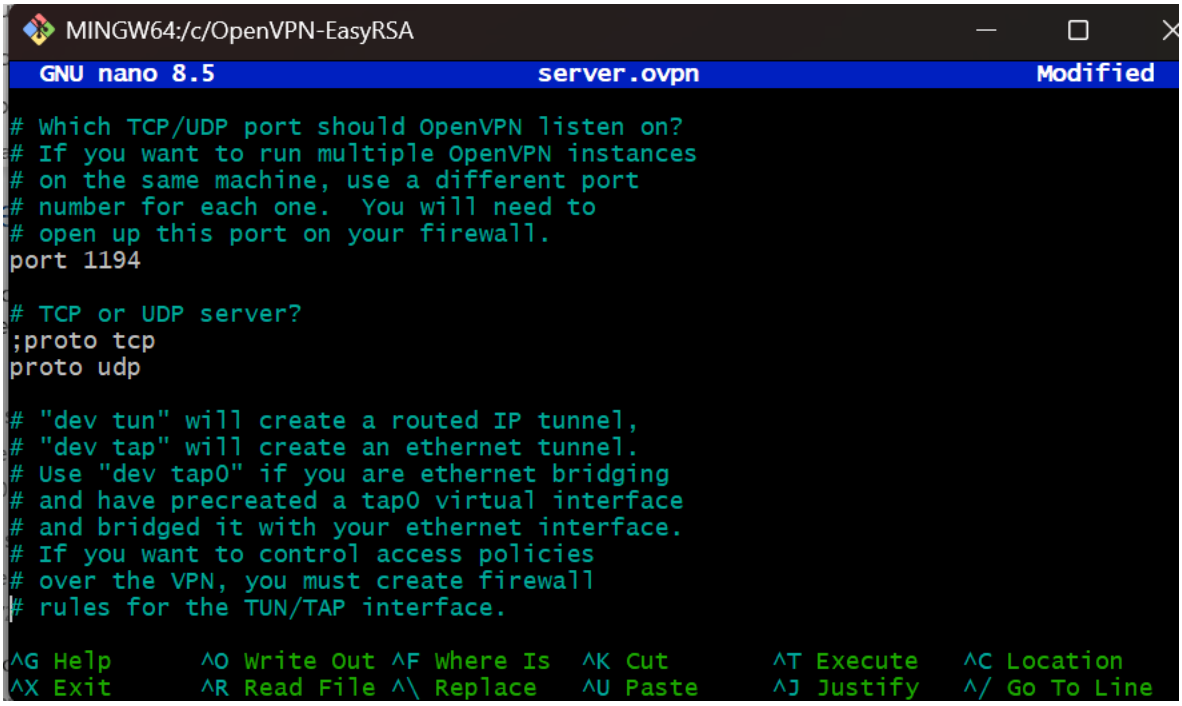
cd /c/EasyRSA-Start

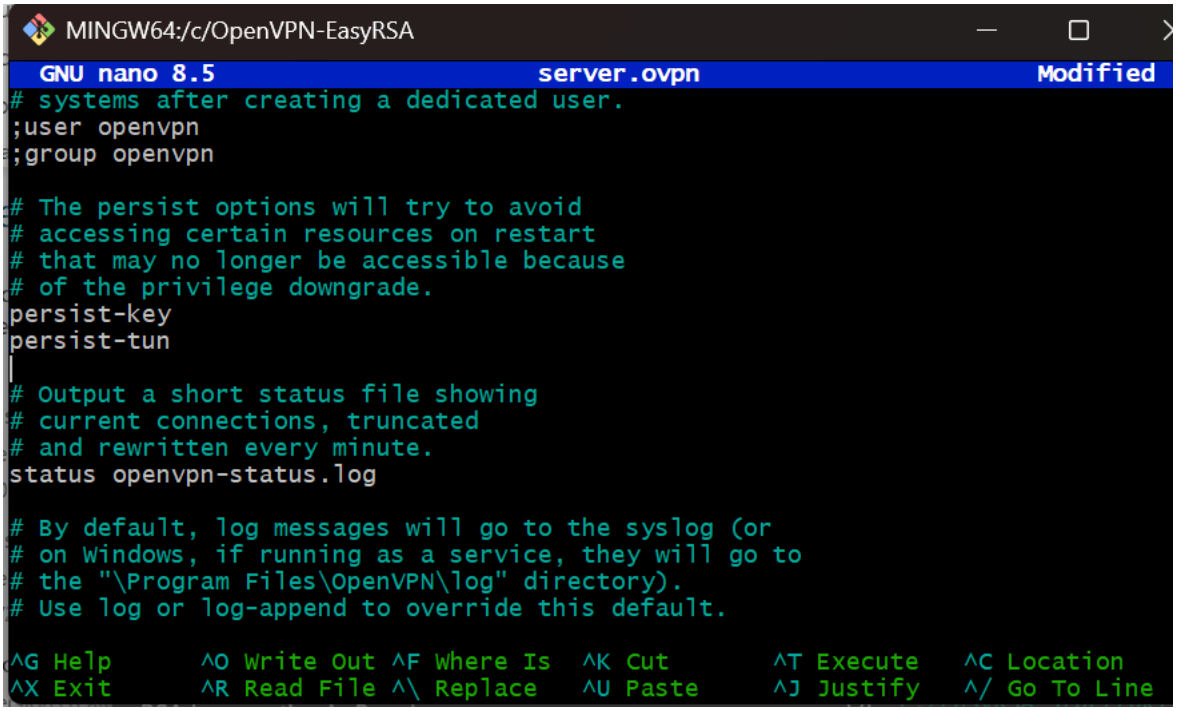
Then, after OpenVPN installation, creating server and client configs using the sample config files in the C:\Program Files\OpenVPN\sample-config folder. I copied the server configuration and copied the client configuration.

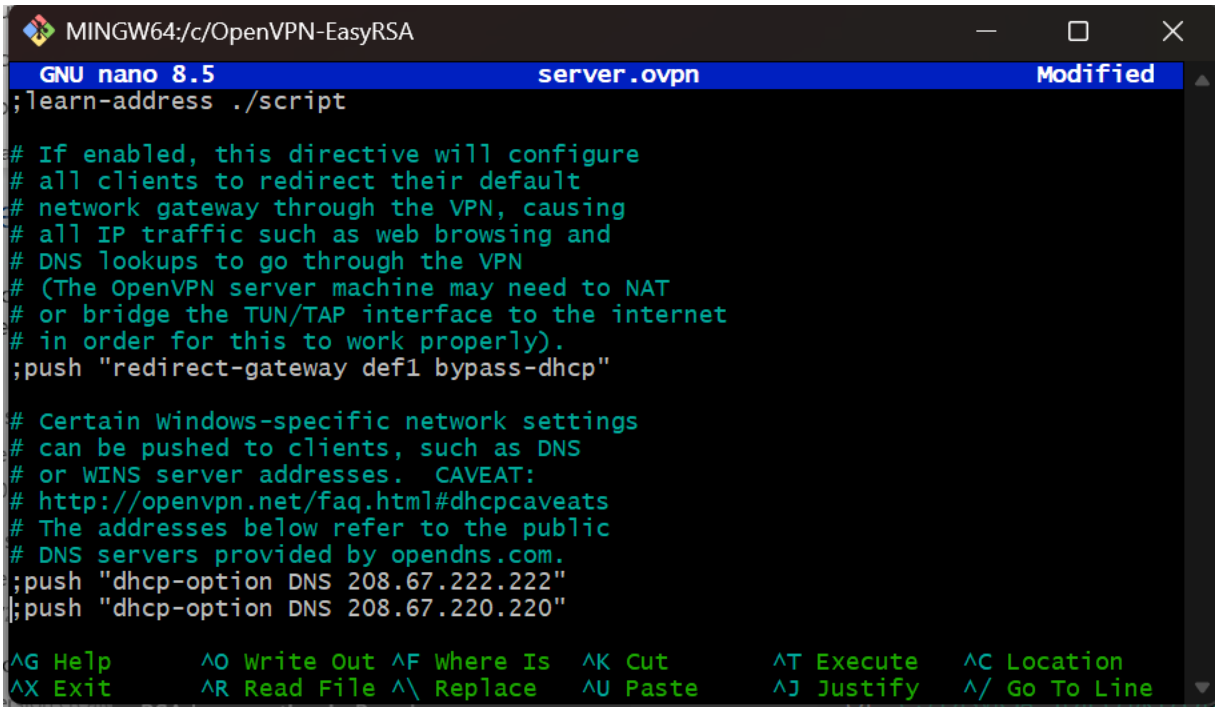
The next step is to open the server config file for editing using nano in Git Bash. It allows to change other necessary settings including port, certificate, key, dh, and tls-auth, so that OpenVPN can run properly in the server environment.

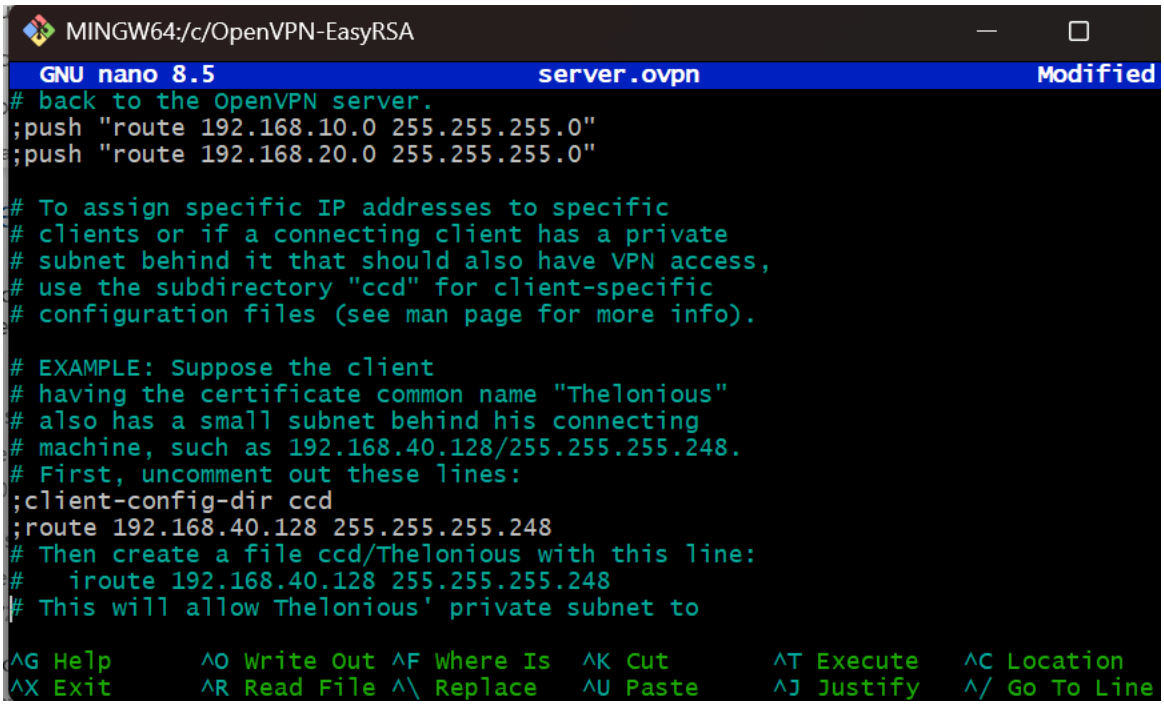
```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ nano server.ovpn
```

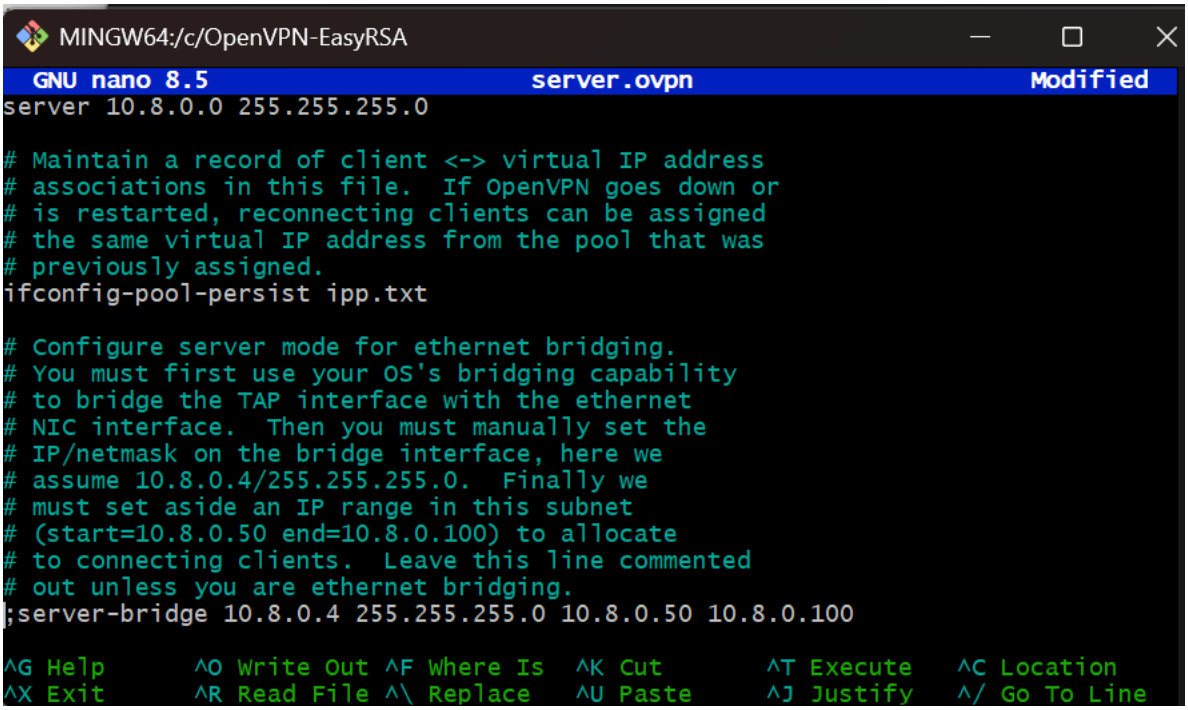
The path to the following line has been fixed (for the server certificate):

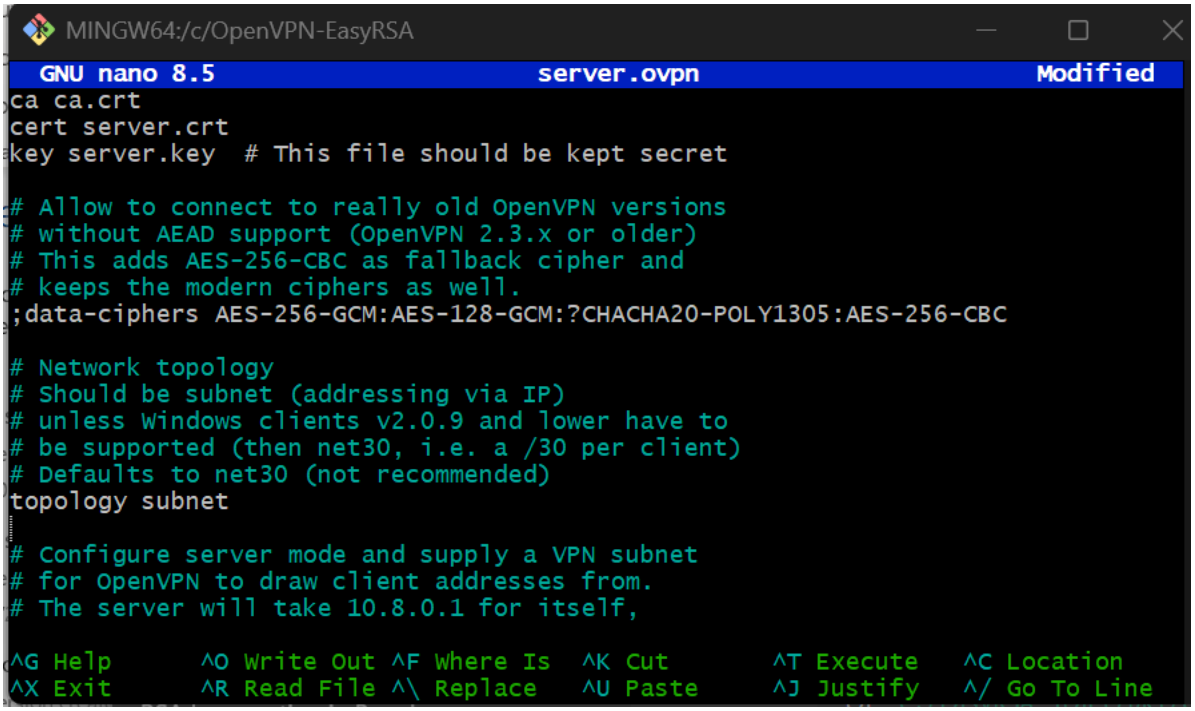


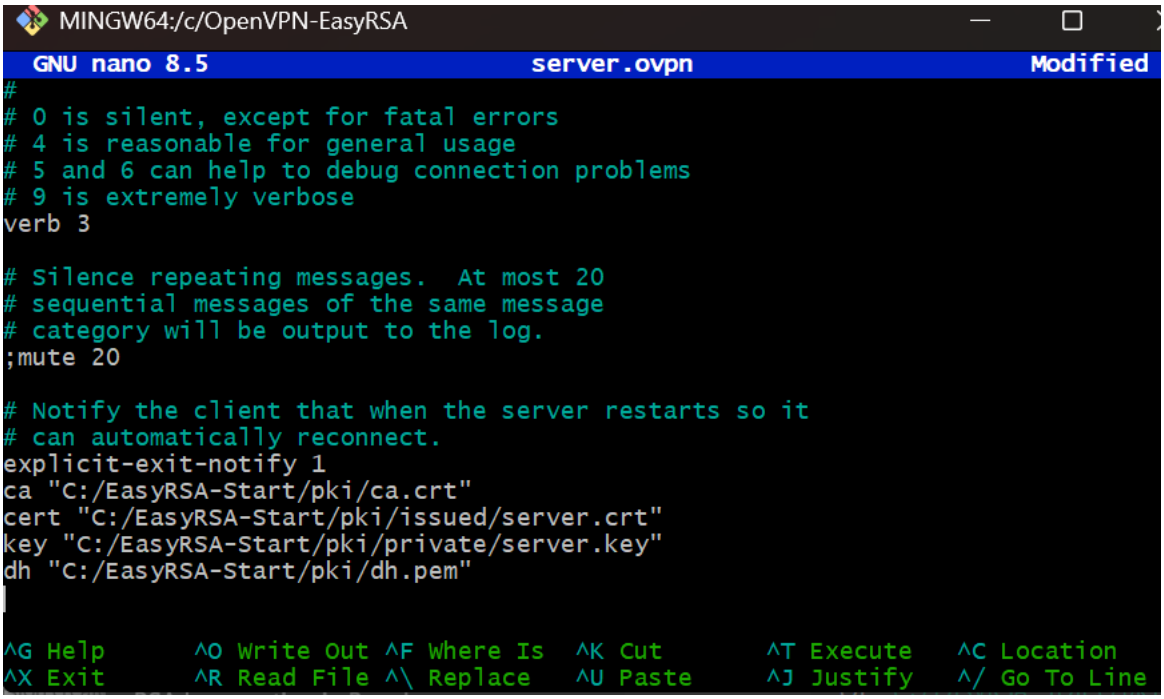

MINGW64:/c/OpenVPN-EasyRSA
GNU nano 8.5 server.ovpn Modified
systems after creating a dedicated user.
;user openvpn
;group openvpn
The persist options will try to avoid
accessing certain resources on restart
that may no longer be accessible because
of the privilege downgrade.
persist-key
persist-tun
Output a short status file showing
current connections, truncated
and rewritten every minute.
status openvpn-status.log
By default, log messages will go to the syslog (or
on Windows, if running as a service, they will go to
the "\Program Files\OpenVPN\log" directory).
Use log or log-append to override this default.
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


MINGW64:/c/OpenVPN-EasyRSA
GNU nano 8.5 server.ovpn Modified
;learn-address ./script
If enabled, this directive will configure
all clients to redirect their default
network gateway through the VPN, causing
all IP traffic such as web browsing and
DNS lookups to go through the VPN
(The OpenVPN server machine may need to NAT
or bridge the TUN/TAP interface to the internet
in order for this to work properly).
;push "redirect-gateway def1 bypass-dhcp"
Certain Windows-specific network settings
can be pushed to clients, such as DNS
or WINS server addresses. CAVEAT:
http://openvpn.net/faq.html#dhcpcaveats
The addresses below refer to the public
DNS servers provided by opendns.com.
;push "dhcp-option DNS 208.67.222.222"
;push "dhcp-option DNS 208.67.220.220"
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


MINGW64:/c/OpenVPN-EasyRSA
GNU nano 8.5 server.ovpn Modified
back to the OpenVPN server.
;push "route 192.168.10.0 255.255.255.0"
;push "route 192.168.20.0 255.255.255.0"
To assign specific IP addresses to specific
clients or if a connecting client has a private
subnet behind it that should also have VPN access,
use the subdirectory "ccd" for client-specific
configuration files (see man page for more info).
EXAMPLE: Suppose the client
having the certificate common name "Thelonious"
also has a small subnet behind his connecting
machine, such as 192.168.40.128/255.255.255.248.
First, uncomment out these lines:
;client-config-dir ccd
;route 192.168.40.128 255.255.255.248
Then create a file ccd/Thelonious with this line:
iroute 192.168.40.128 255.255.255.248
This will allow Thelonious' private subnet to
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


MINGW64:/c/OpenVPN-EasyRSA
GNU nano 8.5 server.ovpn Modified
server 10.8.0.0 255.255.255.0
Maintain a record of client <-> virtual IP address
associations in this file. If OpenVPN goes down or
is restarted, reconnecting clients can be assigned
the same virtual IP address from the pool that was
previously assigned.
ifconfig-pool-persist ipp.txt
Configure server mode for ethernet bridging.
You must first use your OS's bridging capability
to bridge the TAP interface with the ethernet
NIC interface. Then you must manually set the
IP/netmask on the bridge interface, here we
assume 10.8.0.4/255.255.255.0. Finally we
must set aside an IP range in this subnet
(start=10.8.0.50 end=10.8.0.100) to allocate
to connecting clients. Leave this line commented
out unless you are ethernet bridging.
;server-bridge 10.8.0.4 255.255.255.0 10.8.0.50 10.8.0.100
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line





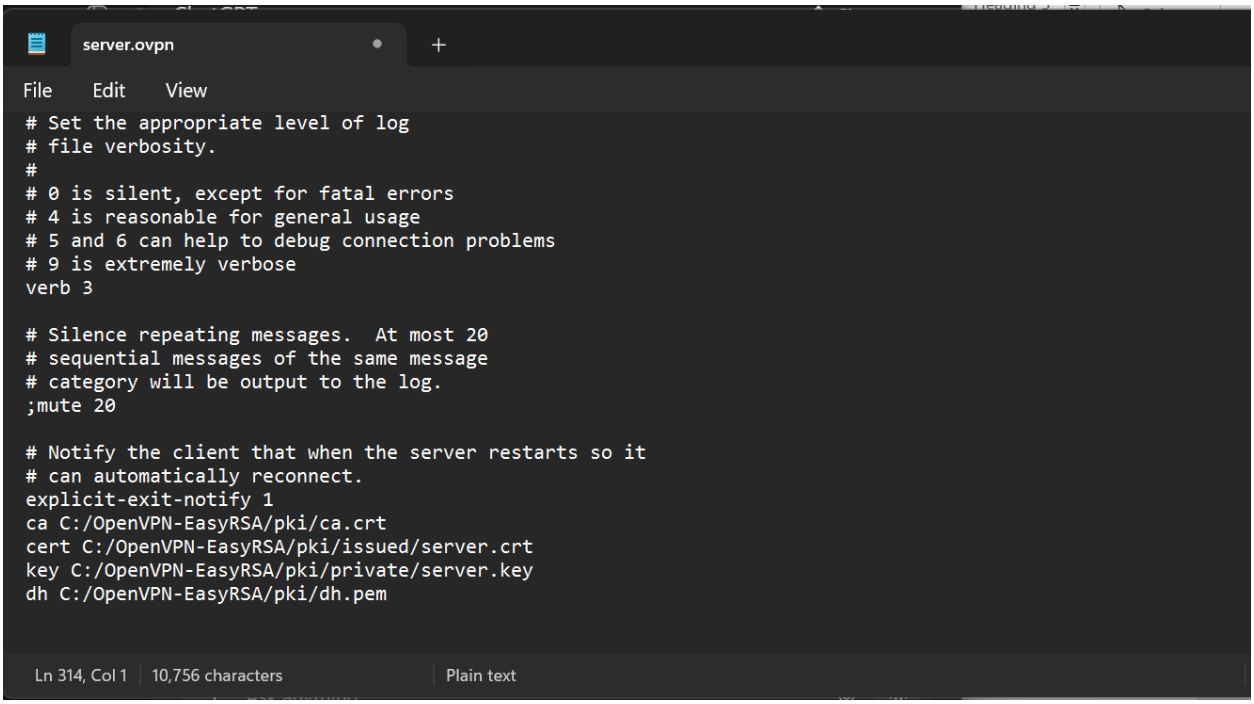
The server configuration file specifies the location of the required certificate and key files. After editing the file, Ctrl + O was pressed to save the file, then Enter was pressed to confirm and Ctrl + X was pressed to exit the editor. This saved the server.ovpn config file and was ready for the OpenVPN server.

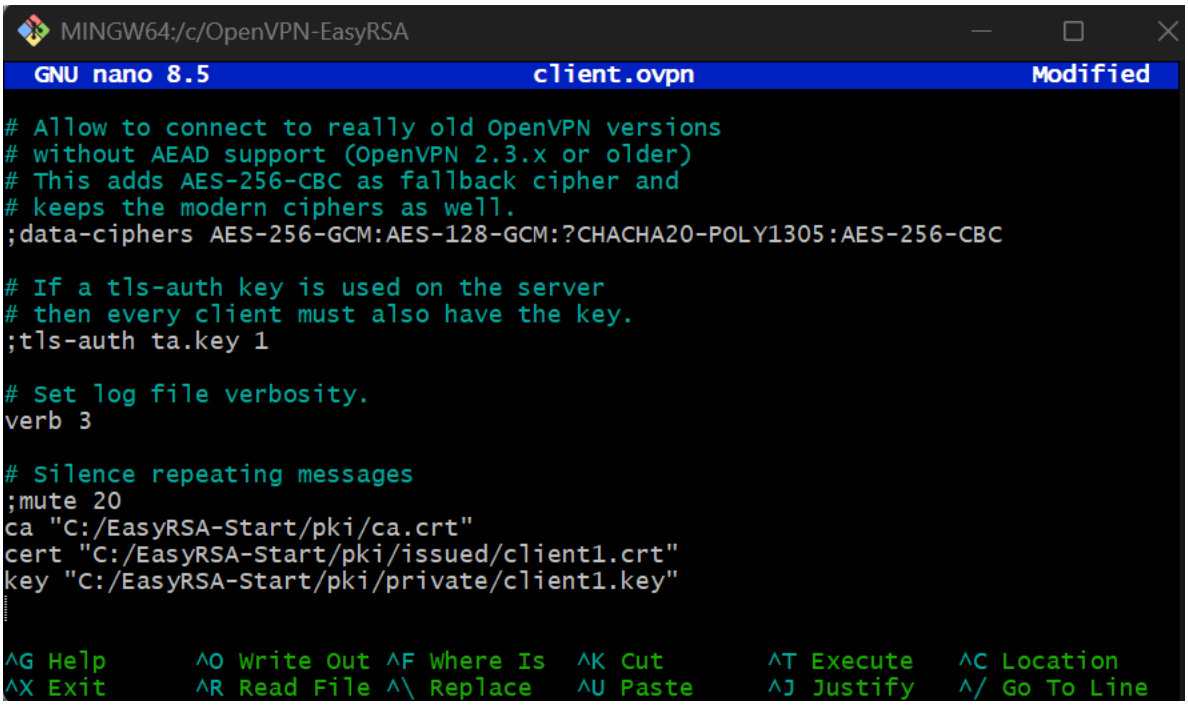
Changing the Client Config:

```
Hasib@Hasib MINGW64 /c/OpenVPN-EasyRSA
$ nano client.ovpn
```

Fixed path to:

By using notepad server.ovpn, it was used to open server.oven file using Notepad. The locations were corrected.

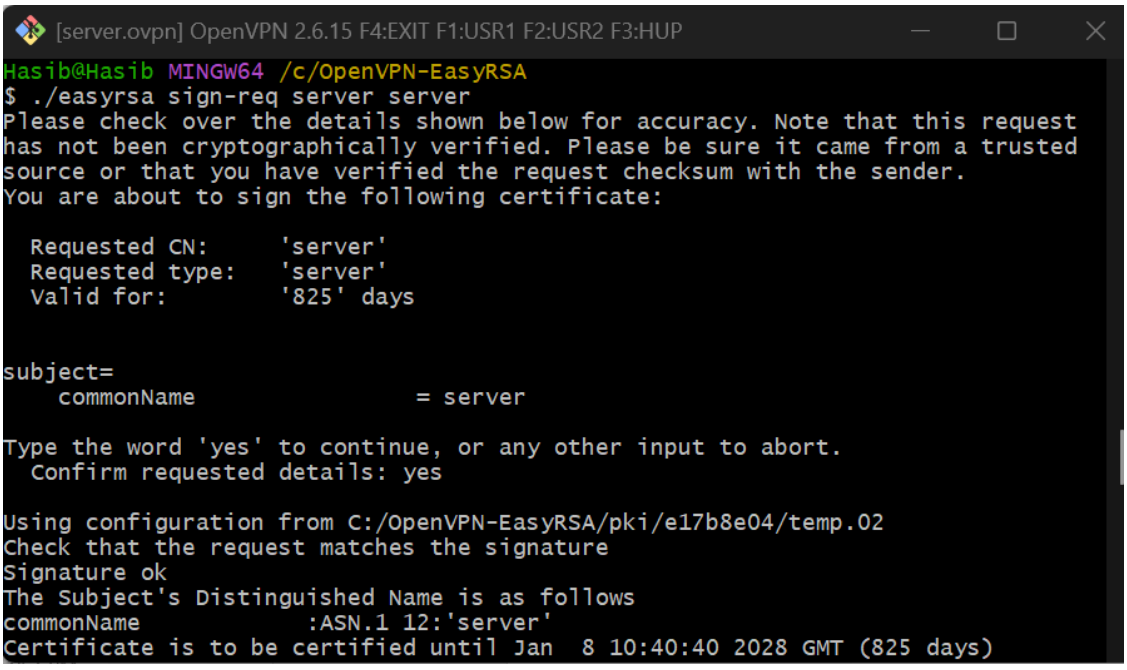


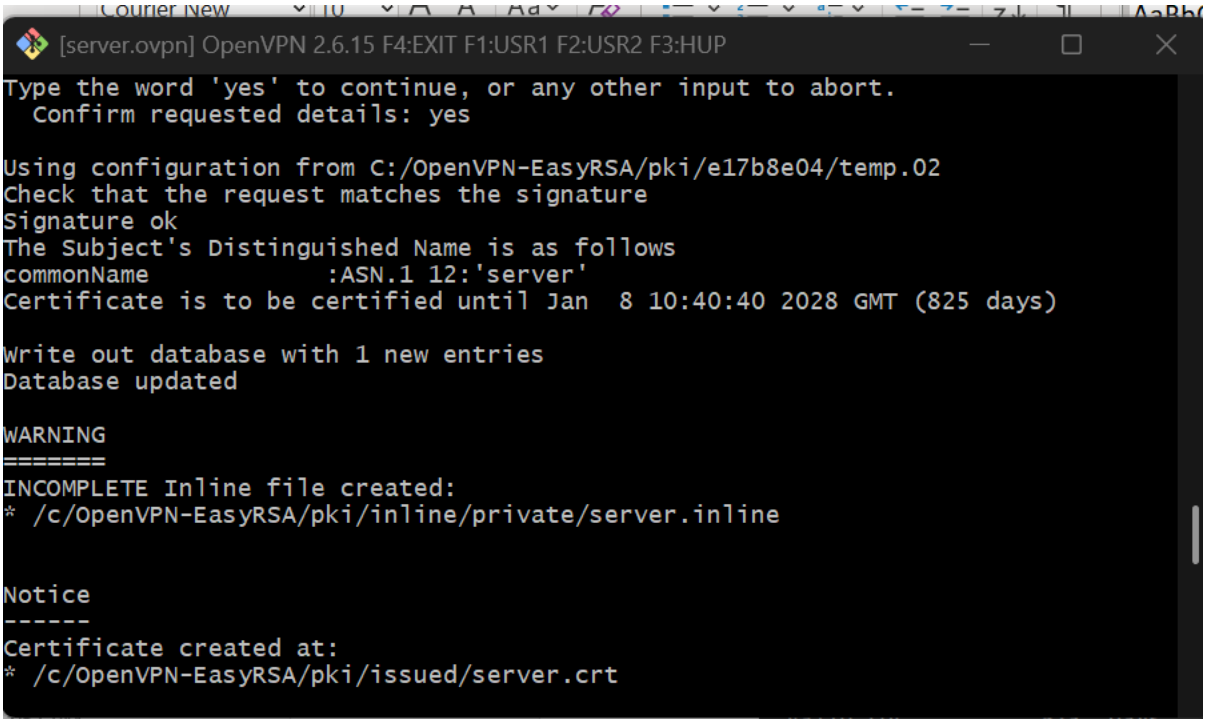


The location of the required certificate and key files was fixed in the client configuration file. After editing, the file was saved with Ctrl + O, Enter was pressed to confirm, and Ctrl + X was used to exit the editor. That created the client.ovpn file and was usable in the OpenVPN client.

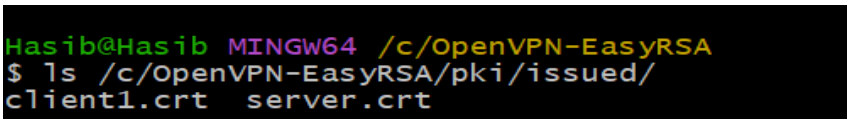
Running OpenVPN Server:

In Git Bash:



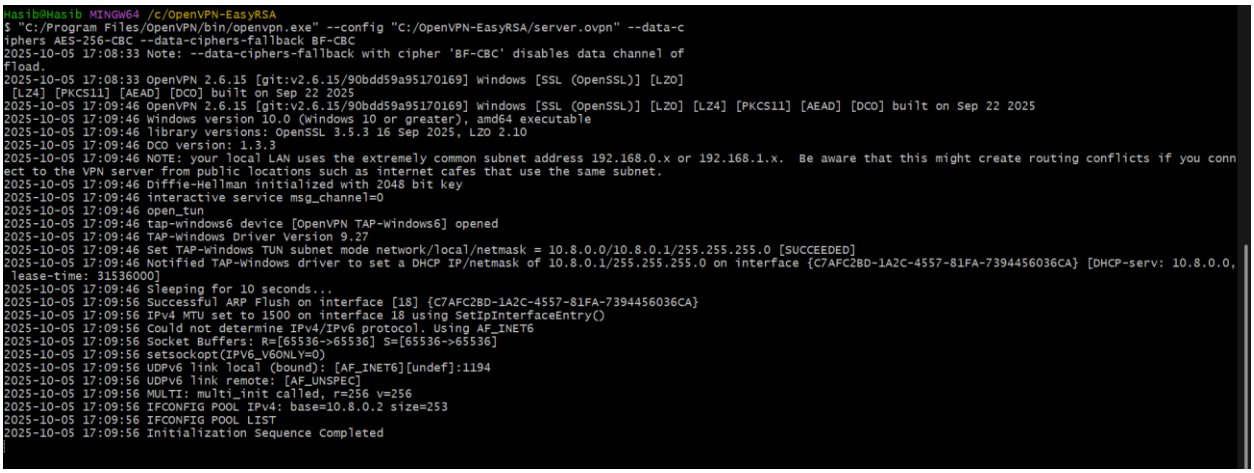


This command listed all files in the /c/OpenVPN-EasyRSA/pki/issued/ directory in Git Bash or the Linux/Unix shell. This usually contained certificate (.crt) files, such as server.crt and client1.crt.



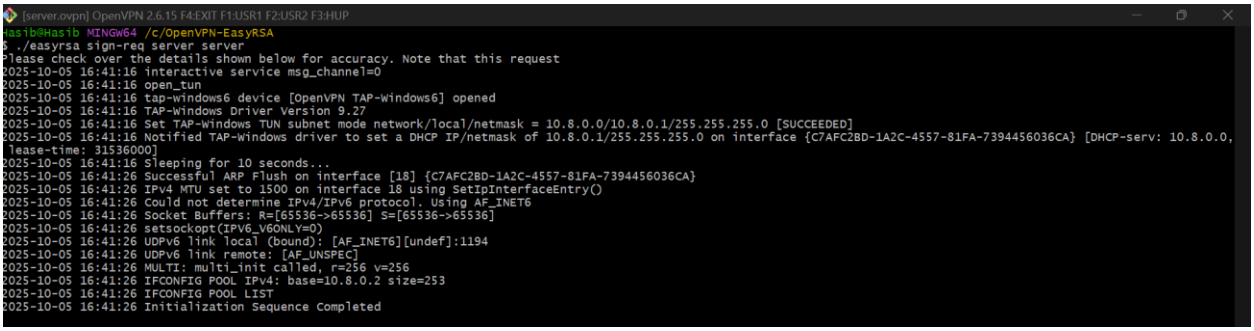
The output showed that there were two certificates in the /c/OpenVPN-EasyRSA/pki/issued/ folder: client1.crt and server.crt, which had already been issued.

The CA was signing correctly, and the necessary authentication materials for OpenVPN were ready.



The OpenVPN server had been successfully started. According to the log output, the Diffie-Hellman key had been initialized to 2048-bit, the TAP-Windows6 device had been created, and the 10.8.0.0/24 subnet had been configured [30]. The UDPv6 link local and remote addresses had been bound, and the IPv4 address range 10.8.0.2-10.8.0.254 had been prepared in the IFCONFIG pool [31]. All routing and DHCP setup had been completed successfully and the “Initialization Sequence Completed” indicated that the server was fully ready [32]. The server log was using OpenVPN version 2.6.15 and the OpenSSL 3.5.3 library [33].

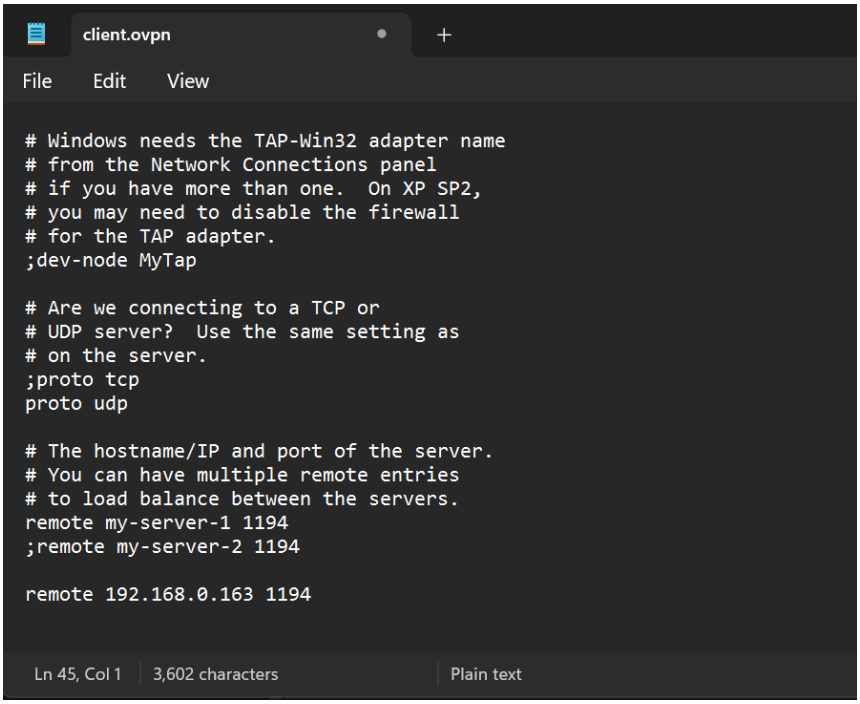
The command /easyrsa sign-req server server signed the server’s CSR using the CA certificate [30], [31].

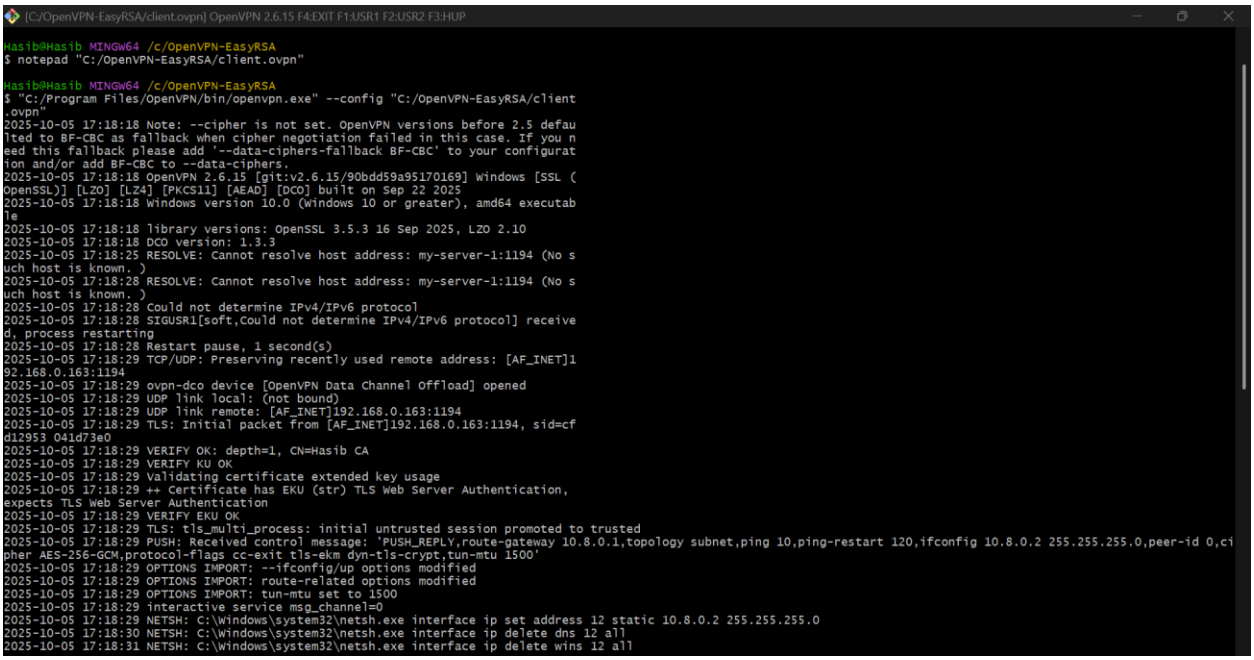


The log showed "Initialization Sequence Completed", the server was running.

Running OpenVPN Client (opens a new Git Bash window):

After checking clint.ovpn using Notepad:





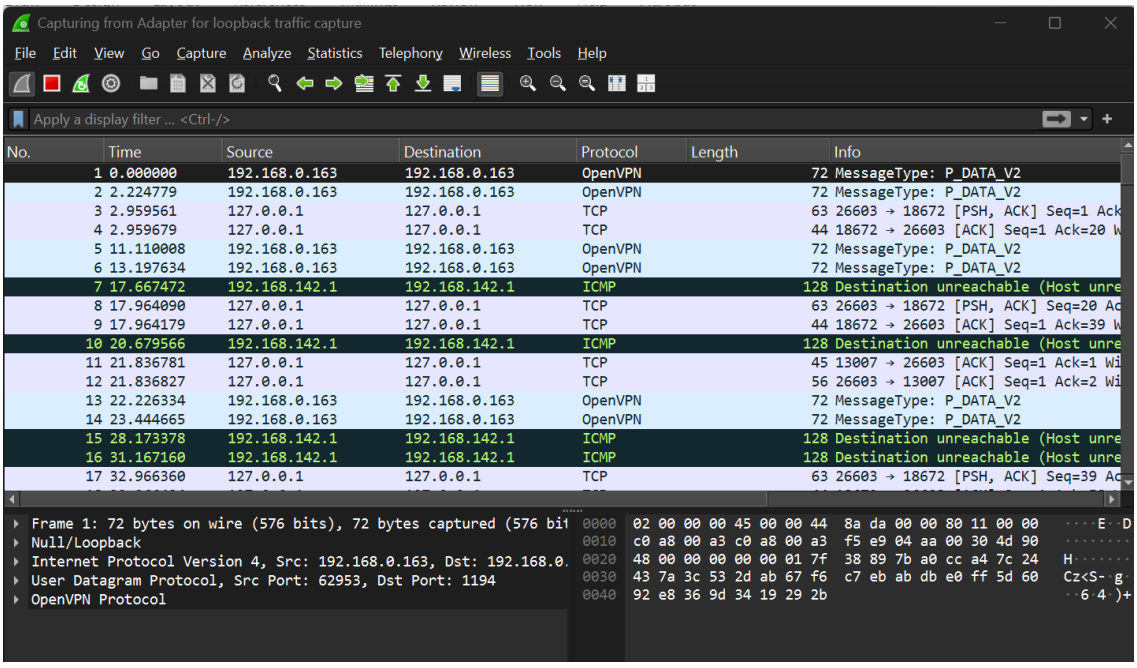
The client was connected because initialization was complete.



3b. Using a traffic sniffer like Wireshark, inspect the interface of the VPN traffic. Show the information about this traffic.

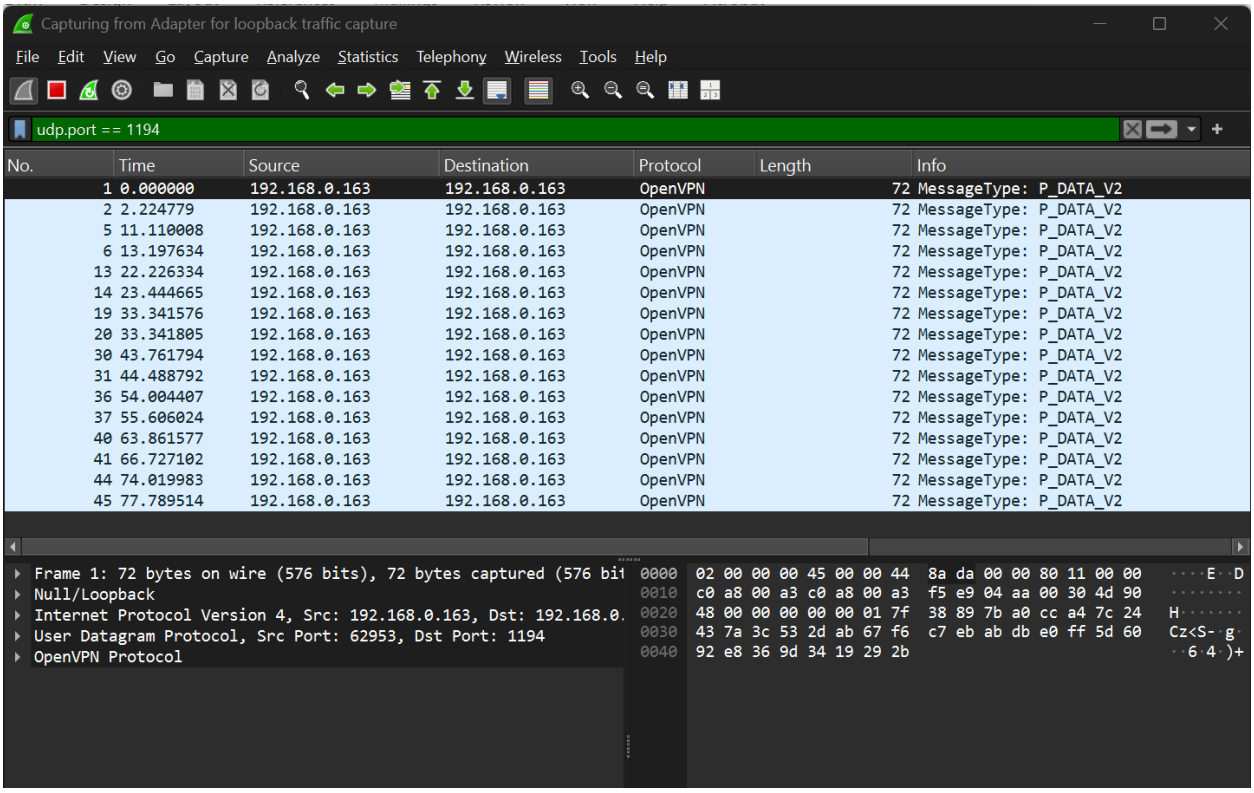
Solution:

After installing and opening Wireshark, the main window on Windows displays Network Interfaces. The TAP-Windows Adapter is typically used for VPN traffic. Double-clicking on that interface starts the capture, and if both the client and server are running, the packets are displayed in real-time. To monitor specific VPN traffic, I applied the filter by typing udp.port == 1194 in the Filter bar, as OpenVPN used UDP port 1194 by default [34], [35].



View traffic information:

It showed Source IP, Destination IP, Protocol, Length, etc. Some encrypted data was visible because OpenVPN used TLS/SSL encryption.



Output:

Source: 192.168.0.163

Destination: 192.168.0.163

Protocol: OpenVPN

Src Port: 62953

Dst Port: 1194

References

[1] “Public key infrastructure”, *Wikipedia.* Available: https://en.wikipedia.org/wiki/Public_key_infrastructure

[2] “What Is Public Key Infrastructure (PKI) & How Does It Work?”, *Okta.* Available: https://www.okta.com/identity-101/public-key-infrastructure/

[3] “Setting Up Your Own Certificate Authority (CA),” *OpenVPN Community Resources*, OpenVPN, 2025. Available: https://openvpn.net/community-resources/setting-up-your-own-certificate-authority-ca/

[4] “OpenVPN Security Overview,” *OpenVPN Documentation*, 2025. Available: https://openvpn.net/community-docs/hardening-openvpn-security.html

[5] “Public Key Infrastructure,” *Wikipedia*, 2025. Available: https://en.wikipedia.org/wiki/Public_key_infrastructure

[6] “What is Public Key Infrastructure (PKI)?,” *Fortinet CyberGlossary*, 2025. Available: https://www.fortinet.com/resources/cyberglossary/public-key-infrastructure

[7] “Public Key Infrastructure (PKI),” *IBM Think*, 2025. Available: https://www.ibm.com/think/topics/public-key-infrastructure

[8] “Root and subordinate certification authorities,” *Microsoft Learn*, 2025. Available: https://learn.microsoft.com/en-us/windows-server/identity/ad-cs/certification-authority-role/

[9] “PKI design considerations using Active Directory Certificate Services,” *Microsoft Learn*, 2025. Available: https://learn.microsoft.com/en-us/windows-server/identity/ad-cs/pki-design-considerations

[10] “Step by Step: 2-Tier PKI Lab,” *Microsoft Community Hub*, 2025. Available: https://techcommunity.microsoft.com/blog/microsoft-security-blog/step-by-step-2-tier-pki-lab/

[11] “Getting Started with EasyRSA,” *EasyRSA Documentation*, 2025. Available: https://easy-rsa.readthedocs.io/en/latest/

[12] “Community OpenVPN Installation and Setup Guide,” *OpenVPN Community Resources*, 2025. Available: https://community.openvpn.net/openvpn/wiki/EasyRSA

[13] “OpenVPN EasyRSA Setup Guide,” *OpenVPN Community Resources*, 2025. Available: https://community.openvpn.net/openvpn/wiki/EasyRSA

[14] “Certificate Authority Best Practices,” *HMS Networks Support*, 2025. Available: https://support.hms-networks.com/docs/overview-certificate-authority-best-practices

[15] “EasyRSA 3 Server Certificate Creation,” *EasyRSA Documentation*, 2025. Available: https://easy-rsa.readthedocs.io/en/latest/

[16] “Generating Diffie-Hellman Parameters,” *HMS Networks Support*, 2025. Available: https://support.hms-networks.com/docs/overview-certificate-authority-best-practices

[17] “EasyRSA 3 File Locations and Output,” *EasyRSA Documentation*, 2025. Available: https://easy-rsa.readthedocs.io/en/latest/

[18] “EasyRSA 3 Server Certificate Creation,” *EasyRSA Documentation*, 2025. Available: https://easy-rsa.readthedocs.io/en/latest/

[19] “OpenVPN EasyRSA File Conflicts,” *OpenVPN Community Resources*, 2025. Available: https://community.openvpn.net/openvpn/wiki/EasyRSA

[20] “OpenVPN EasyRSA Signing Requests,” *OpenVPN Community Resources*, 2025. Available: https://community.openvpn.net/openvpn/wiki/EasyRSA

[21] “OpenSSL x509 Options,” *OpenSSL Documentation*, 2025. Available: https://www.openssl.org/docs/man1.1.1/man1/x509.html

[22] “EasyRSA 3 Documentation - Certificate Expiry,” *EasyRSA Documentation*, 2025. Available: https://easy-rsa.readthedocs.io/en/latest/

[22] “EasyRSA 3 Client Certificate Creation,” *HMS Networks Support*, 2025. Available: https://support.hms-networks.com/docs/overview-certificate-authority-best-practices

[24] “Certificate Expiration Checking with OpenSSL,” *HMS Networks Support*, 2025. Available: https://support.hms-networks.com/docs/overview-certificate-authority-best-practices

[25] “TLS Authentication in OpenVPN,” *OpenVPN Documentation*, 2025. Available: https://openvpn.net/community-resources/how-to/#tls-auth

[26] “OpenVPN Version and Build Information,” *OpenVPN Documentation*, 2025. Available: https://openvpn.net/community-resources/how-to/#check-version

[27] “OpenVPN Server Initialization Logs,” *OpenVPN Documentation*, 2025. Available: https://openvpn.net/community-resources/how-to/#server-logs

[28] “OpenVPN --genkey Secret Option,” *OpenVPN Documentation*, 2025. Available: https://openvpn.net/community-resources/how-to/#tls-auth

[29] “Configuring TLS Authentication in OpenVPN Client,” *OpenVPN Documentation*, 2025. Available: https://openvpn.net/community-resources/how-to/#tls-auth

[30] “Server Startup Log and Configuration,” *OpenVPN Documentation.* Available: https://openvpn.net/community-resources/how-to/

[31] “Managing Certificates and Keys,” *EasyRSA Documentation*. Available: https://easy-rsa.readthedocs.io/

[32] “OpenVPN Server Configuration and Initialization,” *HMS Networks Support.* Available: https://support.hms-networks.com/

[33] “OpenVPN Releases and Source,” *GitHub - OpenVPN Project.* Available: https://github.com/OpenVPN/openvpn

[34] “Getting Started with Wireshark,” Wireshark Documentation. Available: https://www.wireshark.org/docs/wsug_html_chunked/

[35] “OpenVPN UDP Port Usage.” *OpenVPN Documentation.* Available: https://openvpn.net/community-resources/port-forwarding/